

Fluxo

Sistema Bancário Digital

Checkpoint 2 — Contexto e Problema · Requisitos · Diagramas UML · Log de Auditoria

Disciplina	Projetos de Software II
Professor	Marcus Colantoni (marcus.colantoni@uniube.br)
Instituição	UNIUBE — Universidade de Uberaba
Grupo	1
Integrantes	Gabriel Henrique (líder) · João Alfredo (arquiteto/backend) · Harttur Oliveira Pimenta (banco de dados) · Eduardo Henrique (infraestrutura) · Marcus Vinicius (frontend/UX)
Apresentação	17/09/2026
Versão	2.0 — 31/08/2026
Repositório	github.com/AlfredoVentura/Fluxo
Stack	Laravel 11 (PHP 8.2+) · Blade + Tailwind CSS · Flutter · PostgreSQL 16 · Render

Conteúdo deste documento

1. Contexto e Problema (árvore do problema e árvore de objetivos)
2. Levantamento de Requisitos consolidado v2.0 (RF01-RF76, RNF01-RNF44, RN01-RN27)
3. Diagramas de Sequência (SEQ-01 a SEQ-06)
4. Diagramas de Classes (CLS-01 assinatura/retirada de itens · CLS-02 retirada e 2FA)
5. Diagramas de Atividades (ATV-01 a ATV-05)
6. Diagramas de Componentes e de Implantação (CMP-01 e CMP-02)
7. Log de Auditoria (matriz de eventos, modelo de dados, trigger e hash encadeado)

Contexto e Problema

Projeto: Fluxo — Sistema Bancário Digital **Disciplina:** Projetos de Software II — Prof. Marcus Colantoni (UNIUBE) **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09 **Repositório:** <https://github.com/AlfredoVentura/Fluxo> **Versão:** 1.0

1. Identificação do documento

Item	Conteúdo
Nome do produto	Fluxo
Categoria	Banco digital / sistema bancário digital
Problema abordado	Ausência de uma plataforma bancária digital única, transparente e segura, acessível pela web e pelo celular, com rastreabilidade de todas as operações
Público-alvo	Correntistas pessoas físicas (18–45 anos), mobile-first; usuários internos (backoffice, suporte, administrador e auditoria)
Equipe	Grupo 1 — Gabriel Henrique (líder), João Alfredo (arquiteto/backend), Harttur Pimenta (banco de dados), Eduardo Henrique (infraestrutura), Marcus Vinicius (frontend/UX)
Prazo	23/02/2026 a 03/12/2026

2. Contexto

2.1 Cenário externo

A última década transformou o sistema financeiro brasileiro:

- O **Pix**, lançado em 2020, tornou a transferência instantânea e gratuita um serviço esperado por qualquer cidadão — em 2024 foi responsável pela maior parte das transações do país.
- O **Open Finance** reduziu a barreira de entrada para novas instituições e acostumou o cliente a comparar produtos e tarifas.
- Bancos digitais (Nubank, Inter, C6) provaram que é possível abrir conta em minutos, sem agência, com tarifas transparentes e atendimento pelo aplicativo.
- A consequência cultural é clara: **a experiência digital deixou de ser diferencial e passou a ser requisito básico.**

2.2 Cenário acadêmico

Paralelamente, a formação em Engenharia de Software exige que o aluno percorra o ciclo completo — levantamento de requisitos, modelagem UML, implementação, testes, implantação em nuvem, gestão de projeto e documentação — em um objeto de estudo que seja **suficientemente complexo para ser realista e suficientemente delimitado para ser entregue em um semestre.**

Um sistema bancário satisfaz os dois critérios: possui regras de negócio ricas (partidas dobradas, limites, análise de risco, conciliação), requisitos não funcionais exigentes (segurança, integridade, auditoria, LGPD) e uma interface que qualquer avaliador conhece.

2.3 Situação atual (como o problema é resolvido hoje)

Situação	Como o usuário resolve hoje	Limitação
Abrir conta	Comparecimento à agência ou app com espera de dias por análise manual	Lento, depende de horário comercial
Acompanhar gastos	Extrato em PDF ou aplicativo com categorização limitada	Pouca clareza sobre tarifas e lançamentos futuros
Controlar segurança	Ligações para a central, bloqueio por telefone	Baixa autonomia, sem 2FA visível ao cliente
Contestação de lançamento	Formulários e protocolo por telefone	Sem histórico auditável acessível ao cliente
Aprender engenharia de software	Exercícios isolados, sem integração entre modelagem, código, testes e deploy	Baixa transferência para a prática profissional

3. Declaração do problema

O usuário brasileiro não dispõe de uma plataforma bancária digital única, transparente e segura, acessível pela web e pelo celular, que concentre abertura de conta, movimentações, cartões e pagamentos com rastreabilidade completa de cada operação — e a equipe de desenvolvimento não dispõe de um objeto de estudo que permita exercitar o ciclo completo de engenharia de software em um domínio de regras ricas.

3.1 Quadro 5W2H do problema

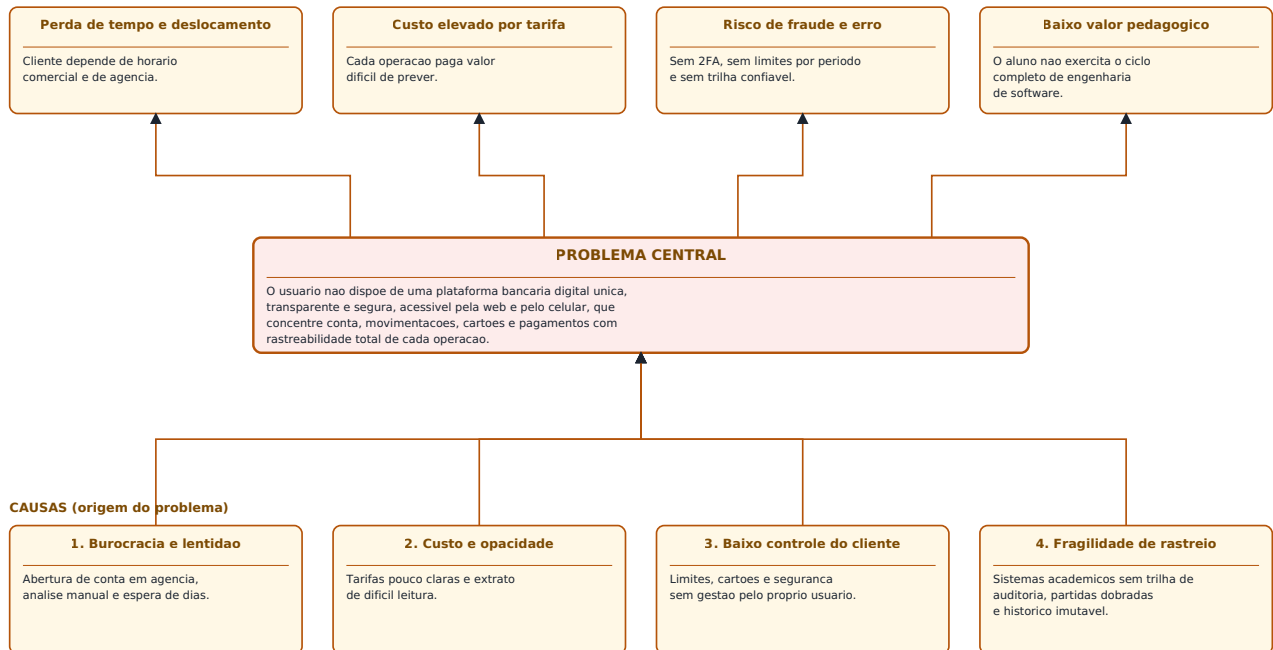
Dimensão	Descrição
What (o quê)	Falta de plataforma bancária digital integrada, transparente e auditável
Who (quem)	Correntista pessoa física mobile-first; usuários internos de backoffice, suporte e auditoria; estudantes de Engenharia de Software
Where (onde)	Qualquer lugar com acesso à internet — o atendimento presencial é o gargalo
When (quando)	No momento de abrir conta, movimentar dinheiro, contestar lançamentos e auditar operações
Why (por quê)	Processos manuais, sistemas sem trilha de auditoria e sem verificação forte de identidade
How (como se manifesta)	Espera por análise manual, tarifas opacas, extrato confuso, ausência de 2FA e de histórico imutável
How much (impacto)	Tempo perdido, custo por tarifa, risco de fraude e de erro contábil, baixo valor pedagógico

4. Árvore do problema

Contexto e Problema — Árvore do Problema do projeto Fluxo

Projeto Fluxo · Grupo 1 · Checkpoint 2 · UNIUBE · Prof. Marcus Colantoni

CONSEQUÊNCIAS (efeitos do problema)



Leitura: as causas (base) produzem o problema central, que gera as consequências (topo).

4.1 Causas

#	Causa	Evidência / justificativa
1	Burocracia e lentidão na abertura de conta	Cadastro presencial e análise manual alongam a abertura por dias
2	Custo e opacidade das tarifas	O cliente descobre o custo depois de contratar, não antes
3	Baixo controle do cliente sobre limites, cartões e segurança	Bloqueio e ajustes dependem de central telefônica
4	Fragilidade de rastreio nos sistemas acadêmicos	Projetos sem partidas dobradas, sem transação ACID e sem log imutável divergem saldo do extrato

4.2 Consequências

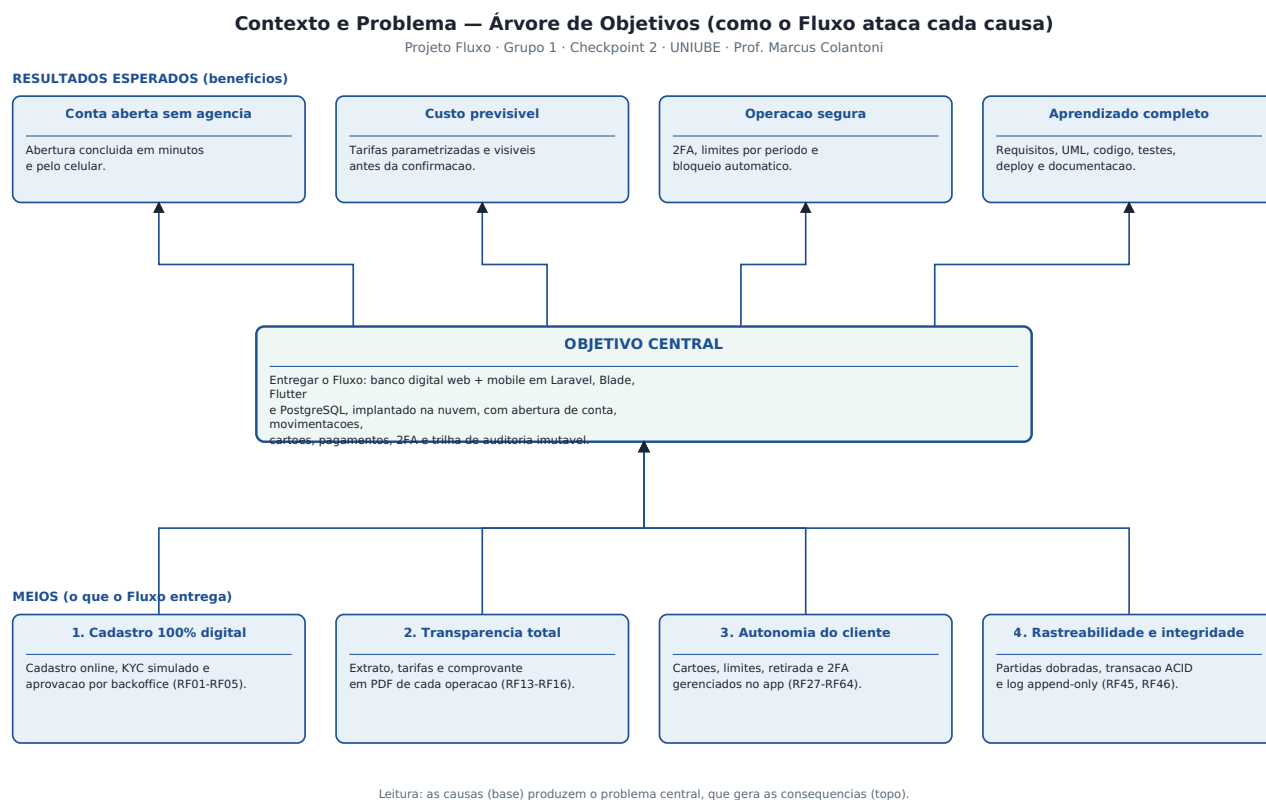
#	Consequência	Métrica que evidencia
1	Perda de tempo e deslocamento	Tempo entre início do cadastro e conta ativa
2	Custo elevado por tarifa	Valor pago em tarifas por mês
3	Risco de fraude e de erro contábil	Operações sem 2FA, sem limite por período e sem trilha confiável
4	Baixo valor pedagógico	Ausência de integração entre modelagem, código, testes, deploy e gestão

5. Stakeholders impactados

Stakeholder	Interesse	Impacto do problema
Cliente (correntista)	Abrir conta rápido, pagar pouco, ter controle	Alto — é o usuário final

Stakeholder	Interesse	Impacto do problema
Favorecido / pagador	Receber transferências e pagar cobranças	Médio
Backoffice	Aprovar cadastros e tratar contestações	Médio — retrabalho manual
Suporte	Atender chamados	Médio — volume de dúvidas sobre taxas e extrato
Administrador	Parametrizar tarifas e limites	Baixo
Auditoria / conformidade	Garantir rastreabilidade e integridade	Alto — sem trilha não há auditoria
Equipe do projeto (Grupo 1)	Aplicar o ciclo completo de engenharia de software	Alto
Professor / avaliador	Verificar requisitos, modelagem, código e implantação	Alto

6. Árvore de objetivos (como o Fluxo ataca cada causa)



Causa	Meio (o que o Fluxo entrega)	Requisito	Resultado esperado
1. Burocracia	Cadastro 100% digital + KYC simulado + aprovação por backoffice	RF01-RF05, RN01	Conta aberta sem ir à agência
2. Opacidade	Extrato detalhado, tarifas parametrizadas e tela de confirmação antes de cada operação	RF13-RF16, RF49, RNF22	Custo previsível
3. Baixo controle	Gestão de cartões, limites, retirada e 2FA pelo próprio app	RF27-RF35, RF53-RF64	Operação segura e autônoma
4. Fragilidade de rastreio	Partidas dobradas, transação ACID, trilha de auditoria imutável com hash encadeado	RF45, RF46, RF48, RN02, RN03, RN27	Saldo sempre igual ao extrato; auditoria possível

7. Solução proposta

Fluxo — banco digital com duas interfaces (web e mobile), backend próprio, banco relacional e implantação em nuvem.

Bloco	Tecnologia	Papel na solução
App mobile	Flutter (Dart)	Operações do cliente no celular: saldo, extrato, Pix, cartões, retirada, 2FA
Web	Blade (Laravel) + Tailwind CSS	Portal do cliente e portal administrativo (backoffice, suporte, auditoria)
Backend	Laravel 11 (PHP 8.2+)	Autenticação, regras de negócio, motor transacional, API REST v1, trilha de auditoria
Banco de dados	PostgreSQL 16	Razão contábil por partidas dobradas, valores em numeric(18,2) , transações ACID
Implantação	Render (plano gratuito)	Deploy automático a partir da branch <code>main</code> do GitHub
Gestão	LibreProject	Cronograma, WBS e Gantt atualizados semanalmente

7.1 Decisões de projeto que respondem diretamente ao problema

1. **Partidas dobradas.** Toda movimentação gera lançamentos de débito e crédito cuja soma é zero; o saldo é calculado, nunca editado. Isso elimina a divergência entre saldo e extrato (causa 4).
2. **Verificação em dois fatores obrigatória em operações de risco.** Login, retirada acima do teto, canal novo e dispositivo não confiável exigem código de uso único (causa 3).
3. **Trilha de auditoria append-only com hash encadeado.** Nenhum registro pode ser alterado ou excluído; correções entram como novo lançamento (causa 4).
4. **Tarifas e limites parametrizados e exibidos antes da confirmação.** O cliente vê o custo antes de contratar (causa 2).
5. **Abertura de conta digital com KYC simulado e fila de aprovação.** Reduz o tempo de abertura de dias para minutos (causa 1).

8. Critérios de sucesso

#	Critério	Meta	Como medir
1	Tempo de abertura de conta	Cadastro submetido em < 5 min; aprovação em até 1 dia útil	Registro de data/hora no banco
2	Convergência saldo x extrato	0 divergências em 100% das operações	Teste automatizado sobre a soma dos lançamentos
3	Cobertura da auditoria	100% das operações sensíveis registradas	Consulta na trilha após bateria de testes
4	Imutabilidade da trilha	0 registros alterados ou excluídos	Trigger + teste de violação
5	Efetividade do 2FA	100% das retiradas acima do teto exigem código	Teste de fluxo + trilha
6	Satisfação do usuário	Usuário novo conclui transferência em ≤ 3 min sem treinamento	Teste com 5 usuários
7	Implantação	Sistema acessível por URL pública no Render	Acesso do professor
8	Processo	Commits semanais de todos os integrantes e cronograma atualizado	Histórico do GitHub e arquivo do LibreProject

9. Restrições, riscos e viabilidade

9.1 Restrições

Tipo	Restrição
Acadêmica	Nenhuma operação financeira real; o sistema é uma simulação
Regulatória	O Fluxo não é instituição autorizada pelo BACEN; Pix, SPI, registradora de boletos e KYC são simulados
Tecnológica	Stack fixada: Laravel + Blade/Tailwind + Flutter + PostgreSQL + Render
Orçamentária	Plano gratuito do Render: serviço hiberna após ~15 min sem tráfego; PostgreSQL gratuito expira em 30 dias
Legal	Adequação à LGPD: minimização, finalidade, mascaramento e direito de exclusão

9.2 Riscos

Risco	Probabilidade	Impacto	Mitigação
Hibernate do Render prejudicar a demonstração	Alta	Médio	"Aquecer" o serviço antes da apresentação; ter vídeo de contingência
Expiração da instância gratuita do PostgreSQL	Alta	Alto	pg_dump semanal versionado; migrations e seeders como fonte de verdade
Divergência entre saldo e extrato	Média	Alto	Partidas dobradas + teste automatizado de consistência
Atraso do cronograma por sobreposição com outras disciplinas	Média	Médio	Cronograma de 32 h semanais no LibreProject, com folga antes de cada checkpoint
Dificuldade de integração Flutter x API	Média	Médio	Contrato de API definido primeiro; monorepo mantém backend e mobile no mesmo PR

9.3 Viabilidade

Dimensão	Análise	Conclusão
Técnica	Stack madura, documentada e já dominada parcialmente pela equipe; Laravel oferece ORM, filas, agendador e autenticação prontos	Viável
Operacional	6 integrantes, ~32 h semanais somadas, cronograma de 95 tarefas até 03/12	Viável
Econômica	Custo zero: Render free tier, PostgreSQL gerenciado gratuito, GitHub, VS Code/ Codespaces e Android Studio gratuitos	Viável
Legal	Dados fictícios, LGPD aplicada por mascaramento e minimização; sem licenças pagas	Viável

10. Rastreabilidade

Item deste documento	Artefato relacionado
Causas 1 e 2	Escopo.md · Requisitos.md (RF01–RF16)
Causa 3	DiagramaClasses.md · DiagramaAtividades.md · RF53–RF64
Causa 4	LogAuditoria.md · RF45, RF46, RF48, RN27
Solução proposta	DiagramaComponentes.md · DiagramaSequencia.md
Cronograma e gestão	QuadroDeGestao.md · libreproject/

Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial: contexto, problema, árvore do problema, árvore de objetivos e critérios de sucesso

4.2 — Levantamento de Requisitos — Versão Consolidada 2.0

Projeto: Fluxo — Sistema Bancário Digital **Disciplina:** Projetos de Software II — Prof. Marcus Colantoni (UNIUBE) **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09 **Repositório:** <https://github.com/AlfredoVentura/Fluxo>

1. Convenções

- **RF** — Requisito Funcional · **RNF** — Requisito Não Funcional · **RN** — Regra de Negócio · **UC** — Caso de Uso.
- **Prioridade (MoSCoW):** **Must** (essencial), **Should** (importante), **Could** (desejável), **Won't** (fora desta versão).
- **Origem:** entrevista simulada com o usuário-tipo, análise de concorrentes (Nubank, Inter, C6), análise documental da disciplina e sessões de cenários de risco.

2. Personas

Persona	Perfil	Necessidades principais	Requisitos atendidos
Ana, 24 anos — correntista mobile-first	Estudante, usa o celular para tudo, renda de R\$ 1.800/mês	Abrir conta sem burocracia, transferir por Pix, controlar limites, pagar boletos, sacar quando precisa	RF01-RF26, RF53-RF64
Carlos, 38 anos — pequeno empreendedor	Tem loja, recebe por Pix e boleto, contrata plano com benefícios	Emitir cobranças, acompanhar entradas, contratar e retirar itens do plano conforme o uso	RF27-RF42, RF65-RF72
Juliana, 32 anos — analista de backoffice	Funcionária interna, aprova cadastros e contestações	Aprovar KYC com rapidez, ver histórico do cliente, agir com rastreabilidade	RF04, RF35, RF43-RF52
Rafael, 45 anos — auditor	Consulta dados, nunca altera	Filtrar e exportar a trilha, comprovar que nada foi adulterado	RF45, RF46, RF48, RF73-RF76

3. Requisitos Funcionais

Módulo 1 — Acesso e Gestão de Conta

ID	Requisito	Prioridade	Casos de uso
RF01	O sistema deve permitir que um visitante se cadastre informando nome completo, CPF, data de nascimento, e-mail, telefone e senha.	Must	UC01
RF02	O sistema deve validar o CPF pelo algoritmo dos dígitos verificadores e recusar CPFs já cadastrados.	Must	UC01
RF03	O sistema deve submeter o cadastro a uma verificação de identidade (KYC) simulada antes de liberar a conta.	Must	UC02
RF04	O sistema deve permitir que o analista de backoffice aprove ou reprove a abertura de conta, registrando o motivo.	Must	UC06

ID	Requisito	Prioridade	Casos de uso
RF05	O sistema deve criar automaticamente uma conta corrente (agência e número únicos) quando o cadastro for aprovado.	Must	UC06
RF06	O sistema deve autenticar o usuário por e-mail e senha, com sessão na web e token na API mobile.	Must	UC03
RF07	O sistema deve permitir ativar a autenticação em dois fatores (2FA) por código enviado ao e-mail.	Should	UC04
RF08	O sistema deve bloquear o acesso após 5 tentativas de login incorretas em 15 minutos.	Must	UC03
RF09	O sistema deve permitir recuperar a senha por link com token de uso único e validade de 30 minutos.	Must	UC05
RF10	O sistema deve permitir ao cliente visualizar e alterar seus dados cadastrais (telefone, e-mail e endereço).	Must	UC07
RF11	O sistema deve exigir confirmação por senha para alterações em dados sensíveis.	Should	UC07
RF12	O sistema deve permitir o encerramento da conta apenas se o saldo for zero e não houver pendências.	Should	UC08

Módulo 2 — Movimentações Financeiras

ID	Requisito	Prioridade	Casos de uso
RF13	O sistema deve exibir o saldo atual da conta, calculado a partir dos lançamentos.	Must	UC09
RF14	O sistema deve exibir o extrato com data, descrição, contraparte, tipo e valor, ordenado do mais recente para o mais antigo.	Must	UC09
RF15	O sistema deve permitir filtrar o extrato por período, tipo de operação e faixa de valor.	Should	UC09
RF16	O sistema deve permitir ao cliente registrar chaves Pix nos tipos CPF, e-mail, telefone e aleatória.	Must	UC10
RF17	O sistema deve impedir o registro de uma chave Pix já vinculada a outra conta.	Must	UC10
RF18	O sistema deve permitir realizar transferência Pix informando a chave do favorecido e o valor.	Must	UC11
RF19	O sistema deve exibir os dados do favorecido para confirmação antes de efetivar a transferência.	Must	UC11
RF20	O sistema deve submeter toda transferência à análise de risco antes da efetivação.	Must	UC12
RF21	O sistema deve permitir transferência interna entre contas Fluxo com liquidação imediata.	Must	UC13
RF22	O sistema deve permitir agendar transferências para data futura, com execução automática.	Should	UC14
RF23	O sistema deve permitir cancelar um agendamento até o dia anterior à execução.	Should	UC14
RF24	O sistema deve permitir depósito simulado via Pix ou boleto, creditando o valor na conta.	Must	UC15
RF25	O sistema deve gerar comprovante em PDF de qualquer transação efetivada, com identificador único.	Should	UC16

ID	Requisito	Prioridade	Casos de uso
RF26	O sistema deve permitir salvar favorecidos frequentes para reuso.	Could	UC13

Módulo 3 — Cartões

ID	Requisito	Prioridade	Casos de uso
RF27	O sistema deve permitir a emissão imediata de cartão virtual, com número, validade e CVV gerados.	Must	UC17
RF28	O sistema deve exibir o número do cartão mascarado, revelando-o apenas mediante reautenticação.	Must	UC17
RF29	O sistema deve permitir solicitar cartão físico e acompanhar o status (solicitado, produção, enviado, entregue).	Should	UC18
RF30	O sistema deve permitir bloquear e desbloquear cartões, de forma temporária ou definitiva.	Must	UC19
RF31	O sistema deve permitir ajustar o limite do cartão respeitando o teto aprovado para o cliente.	Should	UC20
RF32	O sistema deve processar autorizações de compra simuladas, verificando limite e status do cartão.	Should	UC21
RF33	O sistema deve manter a fatura do cartão com lançamentos, data de fechamento e vencimento.	Should	UC22
RF34	O sistema deve permitir contestar um lançamento da fatura informando o motivo.	Could	UC23
RF35	O sistema deve permitir ao backoffice analisar a contestação, deferindo ou indeferindo com justificativa.	Could	UC24

Módulo 4 — Pagamentos e Cobranças

ID	Requisito	Prioridade	Casos de uso
RF36	O sistema deve permitir o pagamento de boleto por linha digitável ou código de barras.	Must	UC25
RF37	O sistema deve validar o dígito verificador do boleto e exibir beneficiário, vencimento e valor antes da confirmação.	Must	UC25
RF38	O sistema deve permitir o pagamento de cobrança Pix por leitura ou colagem de QR Code (payload EMV).	Should	UC26
RF39	O sistema deve permitir ao cliente emitir cobrança Pix com valor, descrição e prazo de validade.	Should	UC27
RF40	O sistema deve permitir agendar pagamentos recorrentes com periodicidade definida.	Could	UC28
RF41	O sistema deve creditar automaticamente o emissor quando uma cobrança for liquidada.	Should	UC29
RF42	O sistema deve notificar o cliente por e-mail e push a cada evento financeiro relevante.	Should	UC30

Módulo 5 — Administração, Segurança e Suporte

ID	Requisito	Prioridade	Casos de uso
RF43	O sistema deve permitir ao administrador criar, editar e desativar usuários internos.	Must	UC31
RF44	O sistema deve implementar controle de acesso por perfil (cliente, suporte, backoffice, administrador, auditor).	Must	UC31
RF45	O sistema deve registrar em trilha de auditoria toda operação sensível, com autor, data/hora, IP e dados alterados.	Must	UC32
RF46	O sistema deve impedir a alteração ou exclusão de registros da trilha de auditoria.	Must	UC32
RF47	O sistema deve permitir ao administrador bloquear contas suspeitas, impedindo novas movimentações.	Must	UC33
RF48	O sistema deve permitir ao auditor consultar a trilha com filtros por autor, período e tipo de operação.	Should	UC34
RF49	O sistema deve permitir parametrizar tarifas, limites por operação e limites diários.	Should	UC35
RF50	O sistema deve gerar relatórios gerenciais de volume transacionado, contas ativas e chamados.	Should	UC36
RF51	O sistema deve permitir ao cliente abrir chamado de suporte com assunto, descrição e anexo.	Should	UC38
RF52	O sistema deve permitir ao analista responder o chamado e alterar seu status.	Should	UC37

Módulo 6 — Verificação em dois fatores (2FA)

ID	Requisito	Prioridade	Casos de uso
RF53	O sistema deve permitir ao cliente ativar e desativar a verificação em dois fatores, exigindo a senha atual e um código válido para desativar.	Must	UC39
RF54	O sistema deve gerar código de verificação de 6 dígitos, de uso único e validade de 5 minutos (3 minutos para retirada), sempre vinculado a uma finalidade (login, retirada ou alteração sensível).	Must	UC40
RF55	O sistema deve armazenar apenas o hash (SHA-256) do código de verificação, nunca o código em texto claro.	Must	UC40
RF56	O sistema deve limitar a 3 tentativas de validação por código, invalidando o desafio e registrando o evento ao exceder.	Must	UC40
RF57	O sistema deve permitir reenviar o código respeitando intervalo mínimo de 60 segundos entre envios.	Should	UC40
RF58	O sistema deve permitir marcar dispositivos como confiáveis por 30 dias, dispensando o 2FA nesses dispositivos, com opção de revogação.	Could	UC41

Módulo 7 — Retirada (saque)

ID	Requisito	Prioridade	Casos de uso
RF59	O sistema deve permitir ao cliente solicitar retirada informando conta de origem, valor e canal (caixa parceiro, agência ou transferência).	Must	UC42

ID	Requisito	Prioridade	Casos de uso
RF60	O sistema deve exigir verificação em dois fatores quando o valor for superior a 30% do limite diário, o canal for inédito, o dispositivo não for confiável ou a operação ocorrer entre 20h e 6h.	Must	UC43
RF61	O sistema deve verificar saldo disponível e limites antes de liberar a retirada, recusando a operação quando insuficiente.	Must	UC42
RF62	O sistema deve gerar um código de retirada de uso único, com validade de 30 minutos, após a liberação da operação.	Must	UC43
RF63	O sistema deve permitir cancelar uma retirada enquanto o código não for utilizado, estornando o valor por lançamento inverso.	Should	UC44
RF64	O sistema deve registrar na trilha de auditoria a solicitação, o desafio de 2FA, a liberação, a utilização e o cancelamento da retirada.	Must	UC43

Módulo 8 — Assinatura e planos

ID	Requisito	Prioridade	Casos de uso
RF65	O sistema deve permitir ao cliente contratar um plano de assinatura entre os planos disponíveis, definindo o dia do vencimento.	Must	UC45
RF66	O sistema deve gerar automaticamente a cobrança mensal da assinatura no dia de vencimento escolhido.	Must	UC48
RF67	O sistema deve permitir fazer upgrade e downgrade de plano, com efeito no ciclo seguinte, preservando a vigência anterior.	Should	UC47
RF68	O sistema deve permitir incluir e retirar itens da assinatura, registrando data e motivo em cada movimentação.	Must	UC46
RF69	O sistema deve manter o histórico dos itens retirados, sem exclusão física do registro.	Must	UC46
RF70	O sistema deve recalcular o valor da cobrança proporcionalmente quando houver inclusão ou retirada de item no meio do ciclo.	Should	UC48
RF71	O sistema deve suspender a assinatura quando a cobrança não for paga após 5 dias do vencimento, bloqueando os itens até a regularização.	Should	UC48
RF72	O sistema deve permitir cancelar a assinatura a qualquer momento, com efeito ao final do ciclo já pago.	Must	UC47

Módulo 9 — Trilha de auditoria ampliada

ID	Requisito	Prioridade	Casos de uso
RF73	O sistema deve registrar em trilha toda tentativa de autenticação, bem-sucedida ou não, com IP, dispositivo e resultado.	Must	UC39, UC40
RF74	O sistema deve registrar a inclusão, a alteração e a retirada de itens da assinatura, com o valor anterior e o novo.	Must	UC46
RF75	O sistema deve permitir ao auditor exportar o resultado da consulta da trilha em CSV.	Should	UC34
RF76	O sistema deve encadear os registros da trilha por hash SHA-256 do registro anterior, permitindo detectar adulteração.	Should	UC34

Resumo da versão 2.0: 76 requisitos funcionais — 24 **Must**, 12 **Should**, 4 **Could** nos módulos novos; 52 requisitos (28 **Must**, 20 **Should**, 4 **Could**) nos módulos 1 a 5.

4. Histórias de usuário e critérios de aceite (módulos novos)

Épico E6 — Verificação em dois fatores

ID	História	Critérios de aceite
US-53	Como cliente quero ativar o 2FA para proteger minha conta mesmo que descubram minha senha.	Dado que estou autenticado, quando ativo o 2FA, então o sistema envia um código de confirmação e passa a exigir o segundo fator nos próximos logins.
US-54	Como cliente quero receber um código de 6 dígitos por e-mail para concluir o login.	Dado credenciais válidas e 2FA ativo, quando informo o código correto em até 5 minutos, então recebo o token de acesso; se o código expirar, o desafio é invalidado.
US-56	Como cliente quero que o código seja invalidado após 3 erros para impedir tentativa forçada.	Dado um desafio ativo, quando erro o código pela terceira vez, então o desafio é cancelado, o evento é registrado na trilha e preciso solicitar novo código.
US-58	Como cliente quero marcar meu celular como confiável para não digitar o código em todo acesso.	Dado que marquei o dispositivo, quando faço login por ele em até 30 dias, então o 2FA é dispensado; posso revogar o dispositivo a qualquer momento.

Épico E7 — Retirada (saque)

ID	História	Critérios de aceite
US-59	Como cliente quero solicitar uma retirada para sacar dinheiro em um caixa parceiro.	Dado saldo suficiente e conta ativa, quando solicito a retirada, então o sistema verifica saldo, limites e se a operação exige 2FA.
US-60	Como cliente quero que retiradas altas exijam confirmação extra para evitar saque indevido.	Dado um pedido acima de 30% do meu limite diário, então o sistema exige o código de verificação antes de liberar o valor.
US-62	Como cliente quero receber um código de retirada para apresentar no caixa parceiro.	Dado que a retirada foi liberada, então recebo um código de 6 dígitos válido por 30 minutos e de uso único.
US-63	Como cliente quero cancelar uma retirada não utilizada para manter o dinheiro na conta.	Dado que o código não foi utilizado, quando cancelo, então o valor é estornado por lançamento inverso e o evento é registrado na trilha.

Épico E8 — Assinatura e retirada de itens

ID	História	Critérios de aceite
US-65	Como cliente quero contratar um plano para pagar menos tarifas.	Dado um plano ativo, quando confirmo a contratação, então a assinatura é criada com vigência imediata e a primeira cobrança é gerada.
US-68	Como cliente quero retirar um item da minha assinatura para parar de pagar por algo que não uso.	Dado um item ativo, quando o retiro informando o motivo, então o registro não é apagado : recebe data e motivo de exclusão, e o histórico permanece consultável.
US-70	Como cliente quero que a cobrança seja proporcional para pagar apenas pelos dias de uso.	Dado que incluí ou retirei um item no meio do ciclo, então a próxima cobrança é recalculada proporcionalmente.
US-72	Como cliente quero cancelar a assinatura para encerrar o serviço quando quiser.	Dado que solicito o cancelamento, então a assinatura permanece ativa até o fim do ciclo já pago e é encerrada depois, com registro na trilha.

Épico E9 — Trilha de auditoria

ID	História	Critérios de aceite
US-73	Como administrador quero registrar todas as tentativas de login para identificar ataques.	Dado um login com senha incorreta, então um registro <code>LOGIN_FALHOU</code> é gravado com IP, dispositivo e horário.
US-75	Como auditor quero exportar a trilha em CSV para analisar os dados fora do sistema.	Dado um filtro aplicado, quando exporto, então o arquivo contém exatamente os registros do filtro, com os mesmos campos da tela.
US-76	Como auditor quero garantir que a trilha não foi adulterada para confiar nela como evidência.	Dado que nenhum registro foi alterado, quando a verificação diária roda, então o hash encadeado é válido em 100% dos registros; caso contrário, um alerta <code>TRILHA_ADULTERADA</code> é gerado.

5. Requisitos Não Funcionais

Classificados pela norma **ISO/IEC 25010** (qualidade de produto de software).

Classificados pela norma **ISO/IEC 25010** (qualidade de produto de software).

RNF — Desempenho e eficiência

ID	Requisito	Métrica de verificação
RNF01	As telas de saldo e extrato devem responder em até 2 segundos no percentil 95.	Teste de carga com 50 usuários simultâneos.
RNF02	As operações de escrita da API (transferência, pagamento) devem responder em até 3 segundos.	Medição de tempo de resposta no log.
RNF03	O sistema deve suportar 100 usuários simultâneos sem degradação perceptível.	Teste com k6 ou JMeter.
RNF04	O aplicativo mobile deve iniciar em até 4 segundos em um aparelho Android de entrada.	Medição em dispositivo real.
RNF05	Consultas de extrato devem usar paginação de no máximo 50 registros por página.	Inspeção de código e da API.

RNF — Segurança

ID	Requisito	Métrica de verificação
RNF06	Toda comunicação entre cliente e servidor deve usar HTTPS (TLS 1.2+).	Verificação do certificado no Render.
RNF07	Senhas devem ser armazenadas com hash bcrypt (custo ≥ 12), nunca em texto claro.	Inspeção da tabela <code>usuarios</code> .
RNF08	Dados sensíveis (CPF, número de cartão) devem ser exibidos mascarados por padrão.	Teste de interface.
RNF09	O sistema deve estar protegido contra as vulnerabilidades do OWASP Top 10 (SQL injection, XSS, CSRF, IDOR).	Checklist OWASP + uso de Eloquent, Blade escaping e tokens CSRF.
RNF10	Tokens de API devem expirar em 60 minutos, com renovação por refresh token.	Teste de expiração.
RNF11	A sessão web deve encerrar após 15 minutos de inatividade.	Teste manual.
RNF12	O sistema deve aplicar rate limiting de 60 requisições por minuto por usuário.	Teste de estresse no endpoint.

ID	Requisito	Métrica de verificação
RNF13	Nenhum dado pessoal deve ser gravado em logs de aplicação.	Revisão de código e amostragem de logs.
RNF14	O sistema deve atender aos princípios da LGPD: finalidade, minimização, consentimento e direito de exclusão.	Checklist de conformidade.

RNF — Confiabilidade e integridade

ID	Requisito	Métrica de verificação
RNF15	Toda movimentação financeira deve ocorrer dentro de uma transação ACID; falha parcial implica rollback total.	Teste de falha induzida no meio da operação.
RNF16	A soma dos lançamentos de uma transação deve ser sempre zero (partidas dobradas).	Teste automatizado sobre o razão.
RNF17	Requisições de transação devem ser idempotentes por chave de idempotência, evitando duplicidade em reenvios.	Teste de reenvio da mesma requisição.
RNF18	Valores monetários devem usar <code>numeric(18,2)</code> no banco e inteiros em centavos na aplicação — nunca ponto flutuante.	Inspeção das migrations e do código.
RNF19	O banco de dados deve ter backup semanal, com procedimento de restauração documentado e testado.	Evidência do dump e teste de restore.
RNF20	O sistema deve ter disponibilidade mensal $\geq 95\%$, considerando as limitações do plano gratuito.	Monitoramento por uptime checker.

RNF — Usabilidade

ID	Requisito	Métrica de verificação
RNF21	Interface responsiva, funcional de 320 px (celular) a 1920 px (desktop).	Teste em três resoluções.
RNF22	Toda operação financeira deve exigir uma tela de confirmação antes da efetivação.	Teste de fluxo.
RNF23	Mensagens de erro devem ser claras, em português, indicando como corrigir o problema.	Revisão do catálogo de mensagens.
RNF24	Um usuário novo deve concluir uma transferência sem treinamento em até 3 minutos.	Teste com 5 usuários reais.
RNF25	O sistema deve atender ao nível AA da WCAG 2.1 em contraste e navegação por teclado.	Auditoria com Lighthouse.

RNF — Manutenibilidade e portabilidade

ID	Requisito	Métrica de verificação
RNF26	O código deve seguir o padrão PSR-12 (PHP) e as convenções do Dart/Flutter.	Análise com <code>Laravel Pint</code> e <code>dart analyze</code> .
RNF27	O backend deve ter cobertura de testes automatizados $\geq 60\%$ nas regras de negócio, desenvolvidas com TDD.	Relatório de cobertura do PHPUnit.
RNF28	Toda alteração de esquema deve ser feita por migration versionada.	Histórico do diretório <code>database/migrations</code> .
RNF29	Configurações sensíveis devem residir em variáveis de ambiente, nunca no repositório.	Inspeção do <code>.gitignore</code> e do <code>.env.example</code> .
RNF30	A API REST deve ser versionada (<code>/api/v1</code>) e documentada em OpenAPI/Swagger.	Acesso à documentação publicada.

ID	Requisito	Métrica de verificação
RNF31	O sistema deve rodar em Linux e ser implantável por deploy automático a partir do GitHub.	Deploy funcional no Render.
RNF32	O aplicativo mobile deve suportar Android 8.0 (API 26) ou superior.	Configuração do <code>build.gradle</code> .
RNF32.1	O código Flutter deve manter compatibilidade com iOS 13+, mesmo que a compilação para iOS não seja demonstrada nesta versão.	Ausência de dependências exclusivas de Android no <code>pubspec.yaml</code> .

RNF — Restrições de projeto

ID	Requisito
RNF33	O backend deve ser desenvolvido em Laravel (PHP 8.2+).
RNF34	O frontend web deve usar Blade com Tailwind CSS.
RNF35	O aplicativo mobile deve ser desenvolvido em Flutter.
RNF36	O banco de dados deve ser PostgreSQL.
RNF37	A hospedagem deve ocorrer no Render, com plano gratuito.
RNF38	O código deve ser versionado no GitHub, com commits semanais e acesso ao professor.
RNF39	Todo o código deve ser desenvolvido durante o semestre, sem reaproveitamento de código legado.

RNF — Auditoria e verificação em dois fatores

ID	Requisito	Métrica de verificação
RNF40	O código de verificação em dois fatores deve ser gerado por gerador criptograficamente seguro (<code>random_int</code>) e armazenado apenas como hash SHA-256.	Inspeção do modelo <code>VerificacaoDoisFatores</code>
RNF41	A gravação da trilha de auditoria não deve acrescentar mais de 100 ms ao tempo de resposta da operação de negócio.	Medição com e sem o job em fila
RNF42	O sistema deve detectar adulteração da trilha pela verificação do hash encadeado, executada diariamente de forma agendada.	Teste de violação induzida
RNF43	Nenhum dado sensível (senha, hash de senha, código 2FA, token de sessão ou CVV) deve constar no payload de auditoria.	Revisão do payload + teste automatizado
RNF44	A trilha de auditoria deve reter os registros por 5 anos para operações financeiras.	Política de retenção documentada

Resumo: 44 requisitos não funcionais em 7 categorias (39 da versão 1.0 + 5 novos).

6. Regras de Negócio

ID	Regra
RN01	Uma conta só é ativada após aprovação do KYC pelo backoffice.
RN02	O saldo de uma conta é sempre a soma dos seus lançamentos — nunca um valor editável diretamente.
RN03	Toda transação gera no mínimo dois lançamentos, cuja soma é zero (partidas dobradas).
RN04	Não é permitido saldo negativo: transações sem saldo suficiente são rejeitadas (não há cheque especial nesta versão).
RN05	O valor mínimo de uma transferência é R\$ 0,01 e o máximo padrão é R\$ 5.000,00 por operação.

ID	Regra
RN06	O limite diário padrão de transferências é R\$ 10.000,00, parametrizável pelo administrador.
RN07	Entre 20h e 6h, o limite por operação cai para R\$ 1.000,00 (limite noturno).
RN08	Um cliente pode ter no máximo 5 chaves Pix, sendo no máximo uma por tipo (exceto aleatória).
RN09	Transferências para favorecido cadastrado há menos de 24 h passam por análise de risco reforçada.
RN10	Transações efetivadas não podem ser excluídas; correções ocorrem por lançamento de estorno.
RN11	Um cartão bloqueado não autoriza compras nem gera lançamentos.
RN12	O limite total dos cartões de um cliente não pode ultrapassar o teto aprovado no cadastro.
RN13	A contestação de um lançamento só pode ser aberta em até 90 dias após a data da compra.
RN14	Uma cobrança Pix expira automaticamente após o prazo de validade definido pelo emissor.
RN15	Contas com saldo diferente de zero não podem ser encerradas.
RN16	Contas bloqueadas por suspeita de fraude não realizam movimentações até liberação pelo administrador.
RN17	Todo acesso ou alteração a dados de terceiros por usuário interno é registrado na trilha de auditoria.

Regras de negócio — versão 2.0 (módulos novos)

ID	Regra
RN18	A retirada exige verificação em dois fatores quando o valor for superior a 30% do limite diário, o canal for inédito, o dispositivo não for confiável ou a operação ocorrer entre 20h e 6h.
RN19	O limite diário de retirada é de R\$ 2.000,00, parametrizável pelo administrador; entre 20h e 6h cai para R\$ 500,00.
RN20	O código de retirada é de uso único e expira em 30 minutos; ao expirar sem uso, o valor retorna à conta por estorno automático.
RN21	Uma assinatura só pode ter um plano ativo por vez; a troca de plano encerra a vigência atual e inicia nova, preservando o histórico.
RN22	A retirada de item da assinatura não exclui o registro: preenche <code>dataExclusao</code> e <code>motivoExclusao</code> , mantendo histórico, valor da fatura do ciclo e trilha de auditoria.
RN23	A cobrança é gerada no dia do vencimento com base nos itens ativos naquela data; inclusões e retiradas no meio do ciclo geram cobrança proporcional.
RN24	O cancelamento da assinatura tem efeito ao final do ciclo já pago; não há reembolso proporcional nesta versão.
RN25	A assinatura é suspensa após 5 dias de atraso, bloqueando os itens até a regularização do pagamento.
RN26	O código de verificação em dois fatores tem uso único, validade de 5 minutos (3 minutos para retirada) e no máximo 3 tentativas.
RN27	Todo registro da trilha de auditoria é encadeado por hash SHA-256 e não pode ser alterado nem excluído; a correção de erro é feita por novo lançamento.

Resumo: 27 regras de negócio (17 da versão 1.0 + 10 novas).

7. Matriz de rastreabilidade (versão 2.0)

Módulo	Requisitos	Casos de uso	Diagramas
1 — Acesso e Gestão de Conta	RF01-RF12	UC01-UC08	UCD-01, SEQ-01, ATV-01
2 — Movimentações Financeiras	RF13-RF26	UC09-UC16	UCD-03, SEQ-03, ATV-02

Módulo	Requisitos	Casos de uso	Diagramas
3 — Cartões	RF27-RF35	UC17-UC24	UCD-04
4 — Pagamentos e Cobranças	RF36-RF42	UC25-UC30	UCD-05, SEQ-05, ATV-04
5 — Administração, Segurança e Suporte	RF43-RF52	UC31-UC38	UCD-07, SEQ-06
6 — Verificação em dois fatores	RF53-RF58	UC39-UC41	SEQ-02, SEQ-04, ATV-03, CLS-02
7 — Retirada (saque)	RF59-RF64	UC42-UC44	SEQ-04, ATV-03, CLS-02
8 — Assinatura e planos	RF65-RF72	UC45-UC48	CLS-01, ATV-05
9 — Trilha de auditoria ampliada	RF73-RF76	UC34, UC39-UC46	SEQ-06, aud-01, CLS-01, CLS-02

7.1 Novos casos de uso incorporados ao diagrama de casos de uso

UC	Nome	Ator principal
UC39	Ativar ou desativar verificação em dois fatores	Cliente
UC40	Validar código de verificação (login ou operação sensível)	Cliente
UC41	Gerenciar dispositivos confiáveis	Cliente
UC42	Solicitar retirada	Cliente
UC43	Confirmar retirada com verificação em dois fatores	Cliente
UC44	Cancelar retirada	Cliente
UC45	Contratar assinatura	Cliente
UC46	Gerenciar itens da assinatura (incluir e retirar)	Cliente
UC47	Alterar ou cancelar plano	Cliente
UC48	Acompanhar cobranças da assinatura	Cliente

Com a inclusão destes 10 casos de uso, o diagrama passa de **38 para 48 casos de uso**, mantendo os 13 atores já documentados.

8. Priorização MoSCoW consolidada

Prioridade	Módulos 1-5	Módulos 6-9	Total
Must	28	17	45
Should	20	6	26
Could	4	1	5
Total	52	24	76

9. Técnicas de levantamento utilizadas

1. **Análise de concorrentes** — Nubank, Inter e C6 Bank, para identificar o conjunto de funcionalidades esperado de um banco digital.
2. **Entrevista simulada** com o usuário-tipo (correntista de 18 a 45 anos, mobile-first).
3. **Análise documental** — material das aulas, requisitos da disciplina (mínimo de 8 entidades, duas interfaces, implantação em nuvem) e checkpoints.
4. **Brainstorming da equipe** com priorização MoSCoW.
5. **Prototipação em papel** das telas principais.

6. **Cenários de risco** — sessão específica para derivar os requisitos de 2FA e de retirada a partir de ameaças (roubo de senha, saque indevido, dispositivo desconhecido).
7. **Matriz de eventos auditáveis** — sessão para derivar RF73-RF76 e RN27 a partir da pergunta "o que um auditor precisaria provar?".

Registro de alterações

Versão	Data	Alteração
1.0	20/08/2026	Levantamento inicial (Checkpoint 1): RF01-RF52, RNF01-RNF39, RN01-RN17
2.0	31/08/2026	Consolidação: inclusão dos módulos de 2FA, Retirada, Assinatura e Auditoria ampliada (RF53-RF76, RNF40-RNF44, RN18-RN27), histórias de usuário com critérios de aceite, novos casos de uso UC39-UC48 e matriz de rastreabilidade

4.3 — Diagramas de Sequência (UML)

Projeto: Fluxo — Sistema Bancário Digital **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09
Repositório: <https://github.com/AlfredoVentura/Fluxo>

1. Convenções adotadas

Elemento	Notação usada
Ator	Caixa com estereótipo «ator» (humano)
Boundary	«boundary» — App Mobile (Flutter) ou Web (Blade)
Control	«control» — Controller da API REST
Entidade / serviço	Caixa sem estereótipo (camada de serviço)
Database	«database» — PostgreSQL
External	«external» — sistemas externos simulados
Mensagem síncrona	Linha cheia com seta fechada
Retorno	Linha tracejada com seta aberta
Auto-mensagem	Laço sobre a própria lifeline
Fragmento	Retângulo tracejado com rótulo alt / loop e guardas entre colchetes
Ativação	Barra estreita sobre a lifeline

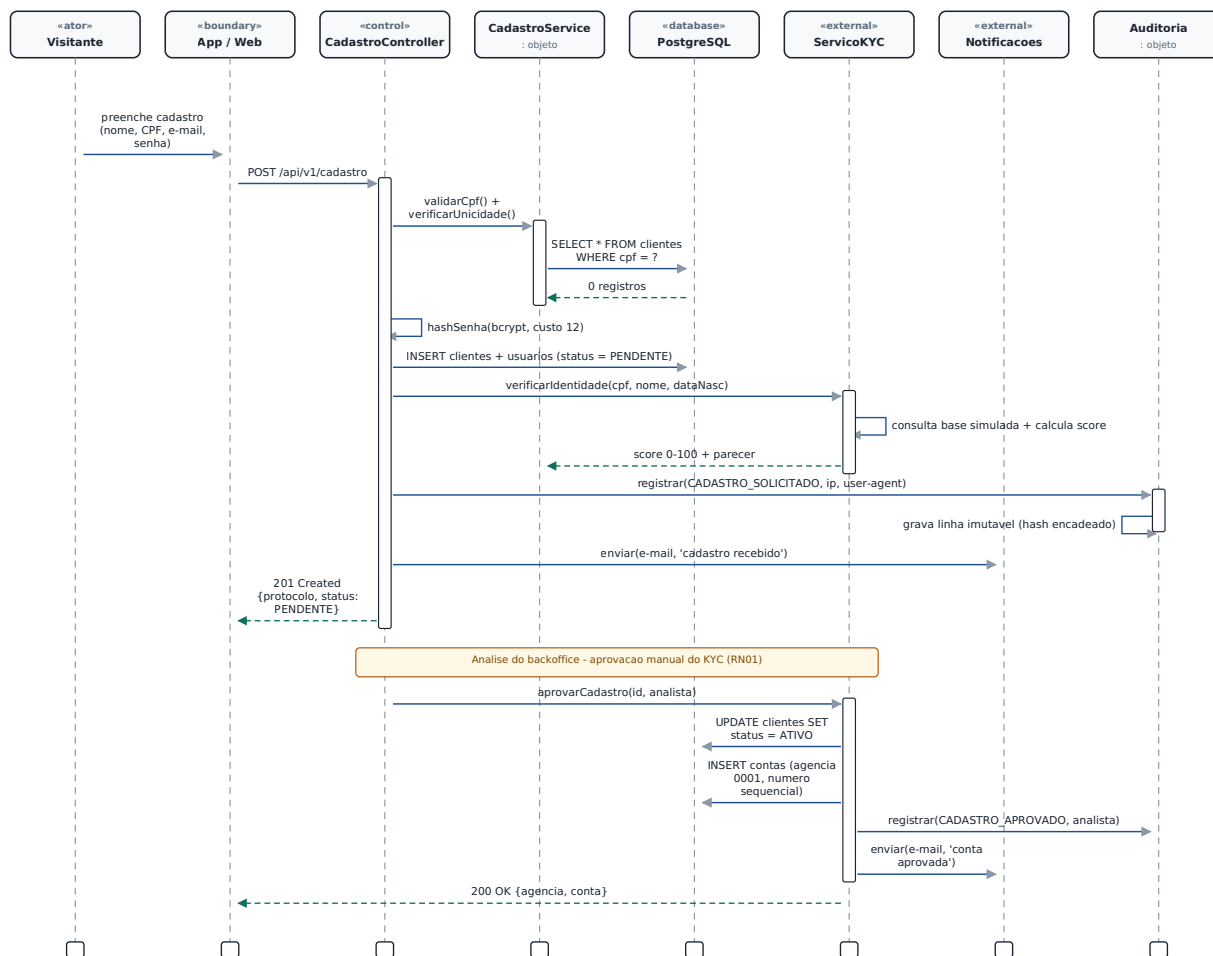
Foram modelados **6 diagramas de sequência**, escolhidos por cobrirem os fluxos de maior risco (dinheiro, identidade e segurança) e por exercitarem os requisitos não funcionais de integridade e auditoria.

ID	Diagrama	Fluxo	Requisitos
SEQ-01	Cadastro, KYC e abertura de conta	Visitante → aprovação do backoffice	RF01-RF05, RF45, RN01
SEQ-02	Autenticação com 2FA	Login → desafio → token	RF06-RF08, RF53-RF58
SEQ-03	Transferência Pix	Consulta de chave → risco → partidas dobradas	RF18-RF21, RN02-RN09
SEQ-04	Retirada com 2FA obrigatório	Solicitação → desafio → liberação	RF59-RF64, RN18-RN20
SEQ-05	Pagamento de boleto	Validação de DV → liquidação simulada	RF36, RF37, RF25
SEQ-06	Trilha de auditoria	Registro (escrita) → consulta (leitura)	RF45, RF46, RF48, RN17

2. SEQ-01 — Cadastro, verificação de identidade (KYC) e abertura de conta

SEQ-01 — Cadastro, verificação de identidade (KYC) e abertura de conta

Projeto Fluxo - Grupo 1 - Checkpoint 2 - RF01-RF05, RF45 - RN01



Participantes

Lifeline	Papel
Visitante	Pessoa ainda não autenticada
App / Web	Boundary — Flutter ou Blade
CadastroController	Recebe POST /api/v1/cadastro, valida a requisição
CadastroService	Regra de negócio: CPF, unicidade, hash de senha, orquestração
PostgreSQL	Persistência
ServicoKYC	Verificação de identidade simulada
Notificacoes	Motor de e-mail e push
Auditoria	Trilha imutável

Pontos relevantes

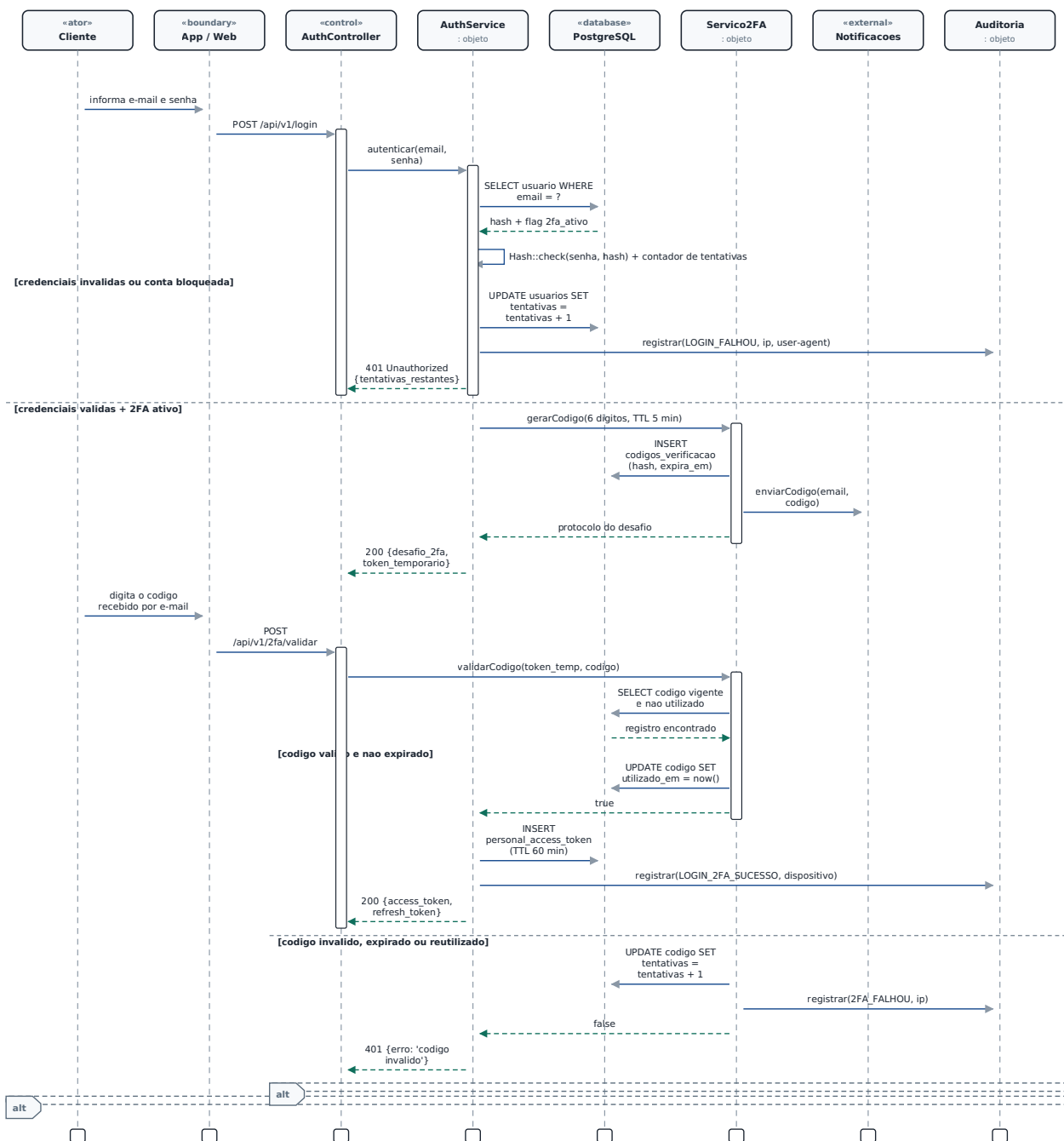
- Validação em duas etapas:** o dígito verificador do CPF é conferido localmente e a unicidade é verificada no banco antes de qualquer inserção (RF02).
- A senha nunca é gravada em texto claro:** Hash::make com bcrypt, custo 12 (RNF07).

3. **A conta nasce inativa:** a linha é criada com `status = PENDENTE` e só é ativada pela aprovação do backoffice (RN01).
4. **Abertura é atômica:** a aprovação grava `clientes.status = ATIVO` e cria a conta (agência + número sequencial) na mesma transação.
5. **Auditoria em dois momentos:** `CADASTRO_SOLICITADO` (pelo visitante) e `CADASTRO_APROVADO` (pelo analista, com identificação do autor).

3. SEQ-02 — Autenticação com verificação em dois fatores (2FA)

SEQ-02 — Autenticação com verificação em dois fatores (2FA)

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF06, RF07, RF08, RF53-RF58 · RNF06, RNF10



Participantes

Cliente · App/Web · AuthController · AuthService · PostgreSQL · Servico2FA · Notificacoes · Auditoria

Pontos relevantes

1. **Contador de tentativas:** 5 falhas em 15 minutos bloqueiam o acesso (RF08); cada falha gera `LOGIN_FALHOU` na trilha.
2. **2FA é condicional:** o desafio só é emitido se `dois_fatores_ativo = true`. Caso contrário, o token é emitido diretamente.
3. **O código nunca é gravado:** apenas o hash (SHA-256) com `expira_em` e contador de tentativas (RF55, RNF40).
4. **Token temporário:** a primeira resposta devolve um token de desafio de curta duração que vincula o usuário ao código gerado — impede que um código seja validado fora de contexto.
5. **Uso único:** ao validar, o registro recebe `utilizado_em` e não pode ser reaproveitado (RN26).
6. **Sessão:** token Sanctum com TTL de 60 minutos (RNF10).

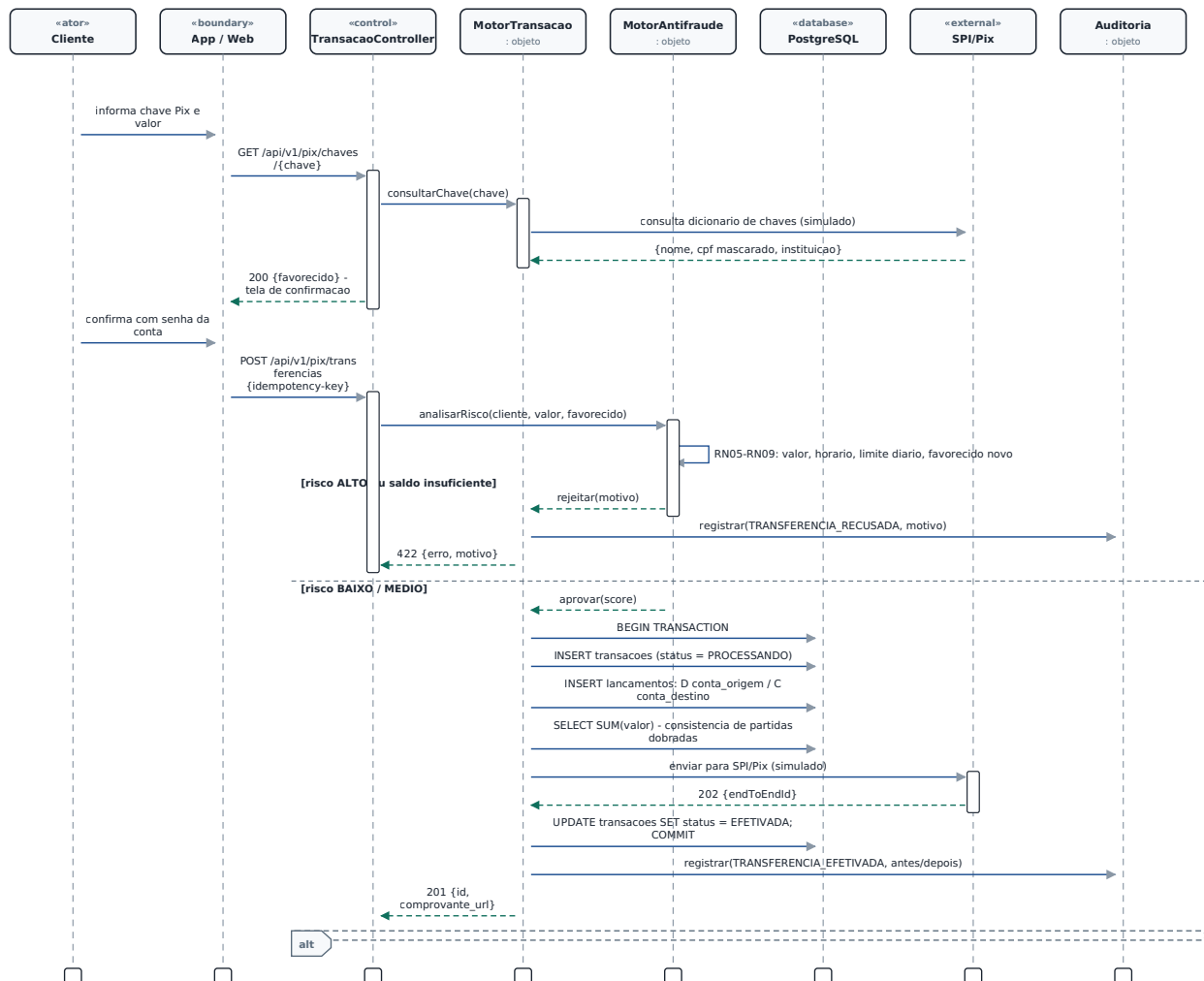
Fragmentos `alt`

- credenciais inválidas ou conta bloqueada → `401` + tentativas restantes + auditoria;
 - código válido e não expirado → emissão do token + `LOGIN_2FA_SUCESSO`;
 - código inválido, expirado ou reutilizado → incremento de tentativas + `2FA_FALHOU` + `401`.
-

4. SEQ-03 — Transferência Pix com análise de risco e partidas dobradas

SEQ-03 — Transferência Pix com análise de risco e partidas dobradas

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF18-RF21 · RN02-RN09 · RNF15-RNF18



Participantes

Cliente · App/Web · TransacaoController · MotorTransacao · MotorAntifraude · PostgreSQL · SPI/Pix (simulado) · Auditoria

Pontos relevantes

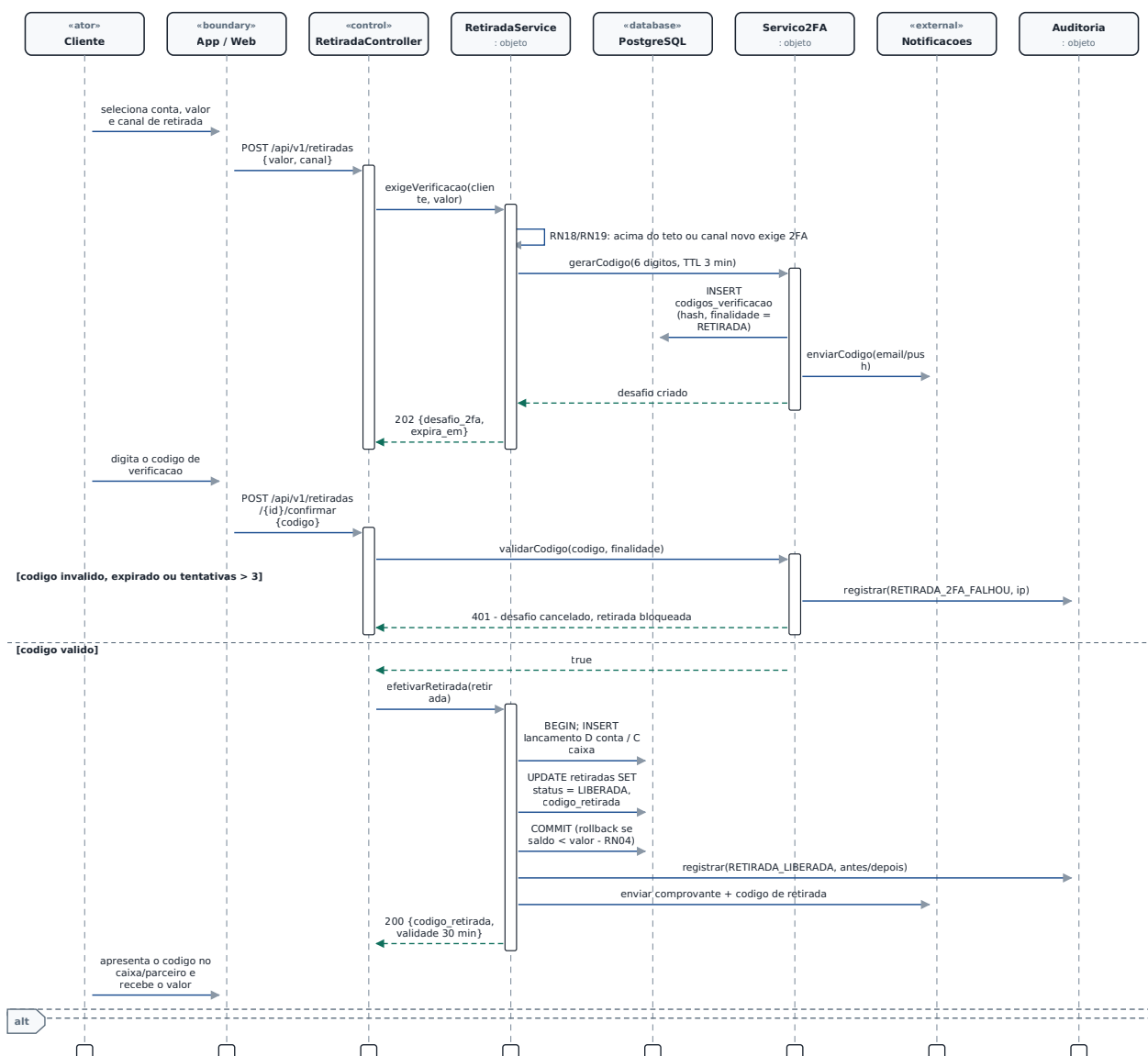
- Confirmação obrigatória:** o sistema consulta a chave, devolve os dados do favorecido e só efetiva após nova confirmação com senha (RNF22, RF19).
- Idempotência:** o cabeçalho `Idempotency-Key` impede que um reenvio duplique a transferência (RNF17).
- Análise de risco antes do débito:** o `MotorAntifraude` aplica RN05 a RN09 (valor por operação, horário noturno, limite diário, favorecido novo, favorecido cadastrado há menos de 24 h).
- Transação ACID:** `BEGIN` → `INSERT transacoes` → `INSERT lancamentos` (débito na origem, crédito no destino) → verificação da soma → `COMMIT`. Falha parcial implica `ROLLBACK` total (RNF15, RNF16).
- Partidas dobradas:** a soma dos lançamentos de uma transação é sempre zero; o saldo é derivado, nunca editado (RN02, RN03).

6. **Auditoria com antes/depois:** o registro guarda o estado anterior e o novo em `jsonb`.

5. SEQ-04 — Retirada (saque) com verificação em dois fatores obrigatória

SEQ-04 – Retirada (saque) com verificação em dois fatores obrigatória

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF59-RF64 · RN18-RN20 · RF45



Participantes

[Cliente](#) ·
 [App/Web](#) ·
 [RetiradaController](#) ·
 [RetiradaService](#) ·
 [PostgreSQL](#) ·
 [Servico2FA](#) ·
 [Notificacoes](#) ·
 [Auditoria](#)

Regra central (RN18)

A retirada exige 2FA quando **qualquer** das condições abaixo for verdadeira:

- valor superior a **30% do limite diário** do cliente;
- canal de retirada ainda **não utilizado** por aquele cliente;
- **dispositivo não confiável**;

- operação realizada entre **20h e 6h**.

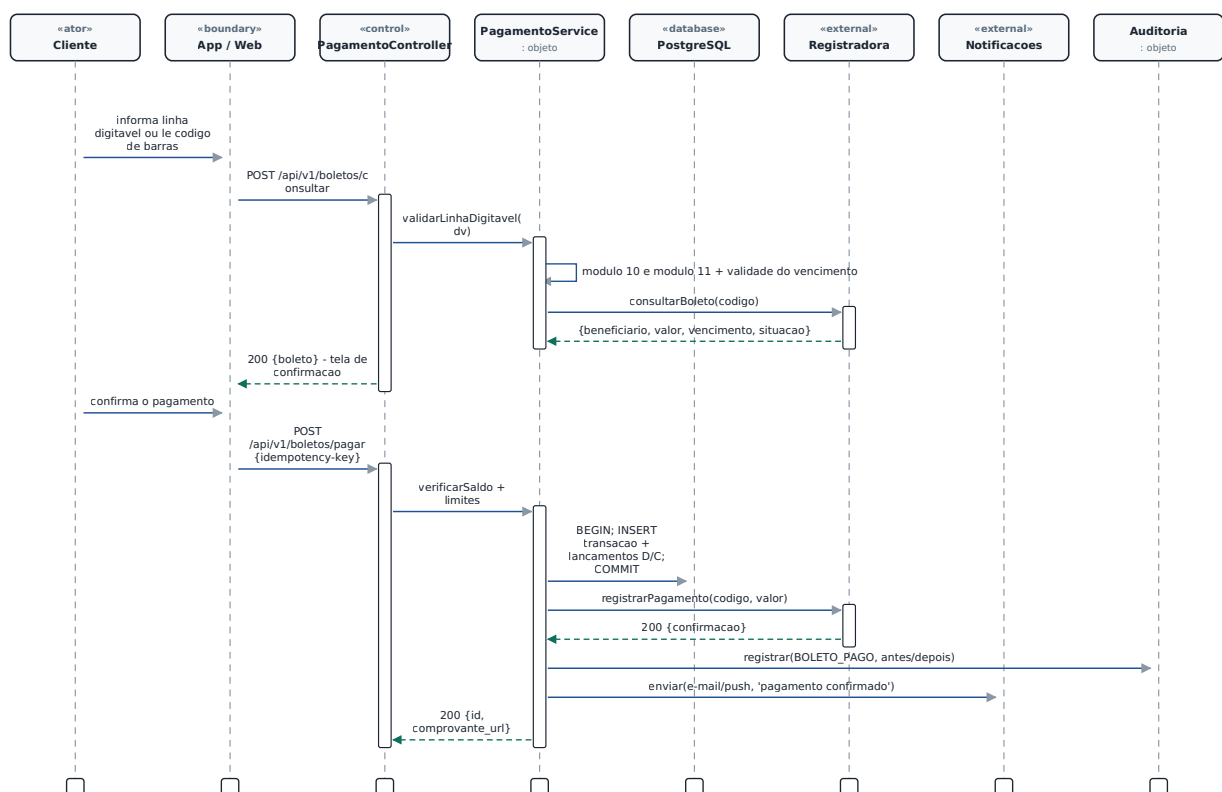
Pontos relevantes

1. O desafio é criado com `finalidade = RETIRADA`, o que impede que um código de login seja reaproveitado para sacar.
2. O código da retirada expira em **3 minutos** (mais curto que o de login) e admite no máximo **3 tentativas** (RN26).
3. Após 3 tentativas o desafio é cancelado e a retirada é bloqueada, com registro `RETIRADA_2FA_FALHOU`.
4. A efetivação só ocorre dentro de transação: `INSERT` do lançamento de débito na conta e crédito na conta caixa, com `ROLLBACK` automático se o saldo for insuficiente (RN04).
5. Liberada a retirada, o sistema gera um **código de retirada de uso único**, válido por 30 minutos (RN20). Se não for utilizado, um job de expiração estorna o valor automaticamente.
6. Todo o percurso — solicitação, falha de 2FA, liberação e cancelamento — é registrado na trilha de auditoria.

6. SEQ-05 — Pagamento de boleto com validação e liquidação simulada

SEQ-05 — Pagamento de boleto com validação e liquidação simulada

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF36, RF37, RF25 · RN04, RN10



Participantes

Cliente · App/Web · PagamentoController · PagamentoService · PostgreSQL · Registradora (simulada) · Notificacoes · Auditoria

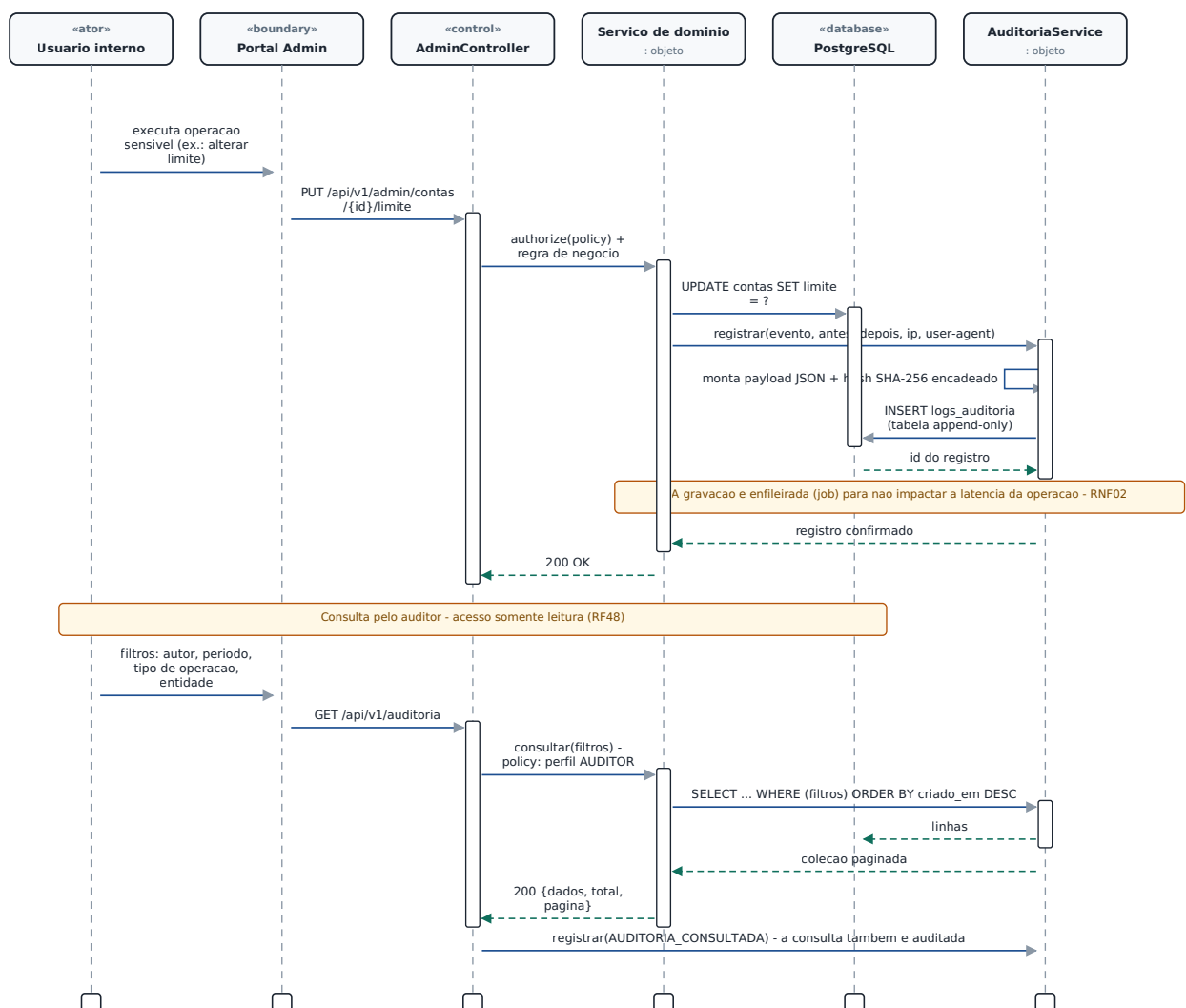
Pontos relevantes

1. **Validação local antes da consulta:** módulo 10 e módulo 11 sobre a linha digitável evitam requisições inúteis à registradora (RF37).
2. **A registradora é consultada duas vezes:** uma para obter beneficiário, valor, vencimento e situação; outra para registrar a liquidação.
3. O débito só acontece após confirmação explícita e verificação de saldo.
4. A operação é idempotente por chave de idempotência e gera comprovante em PDF (RF25).
5. Evento `BOLETO_PAGO` gravado na trilha com o payload completo (antes/depois).

7. SEQ-06 — Registro e consulta da trilha de auditoria

SEQ-06 — Registro e consulta da trilha de auditoria (log imutável)

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF45, RF46, RF48 · RN17



Participantes

Usuário interno · Portal Admin · AdminController · Serviço de domínio · PostgreSQL · AuditoriaService

Pontos relevantes

1. **Escrita desacoplada:** o serviço de domínio publica o evento e a gravação é feita por um job em fila, para não acrescentar latência à operação de negócio (RNF41).
2. **Hash encadeado:** cada registro carrega o SHA-256 do registro anterior, permitindo detectar remoção ou adulteração (RF76).
3. **Tabela append-only:** um trigger no PostgreSQL bloqueia `UPDATE` e `DELETE` (RF46).
4. **A consulta também é auditada:** o auditor tem acesso somente leitura, mas o ato de consultar gera `AUDITORIA_CONSULTADA` (RN17).
5. **Política de acesso:** a policy do Laravel garante que apenas o perfil `AUDITOR` (e o `ADMINISTRADOR`) alcancem o endpoint `GET /api/v1/auditoria` (RF44, RF48).

8. Rastreabilidade requisito × diagrama de sequência

Requisito	SEQ-01	SEQ-02	SEQ-03	SEQ-04	SEQ-05	SEQ-06
RF01–RF05 (cadastro/KYC)	☐					
RF06–RF08 (login/bloqueio)		☐				
RF13–RF16 (saldo/extrato/chaves)			☐			
RF18–RF21 (Pix)			☐			
RF25 (comprovante PDF)			☐		☐	
RF36, RF37 (boleto)					☐	
RF45, RF46, RF48 (auditoria)	☐	☐	☐	☐	☐	☐
RF53–RF58 (2FA)		☐		☐		
RF59–RF64 (retirada)				☐		
RN02, RN03 (partidas dobradas)			☐	☐	☐	
RN17 (acesso a dados de terceiros)						☐
RNF15–RNF17 (ACID e idempotência)			☐	☐	☐	
RNF41 (latência da auditoria)						☐

9. Como estes diagramas foram produzidos

Os SVGs são gerados por script (`scripts/gen_seq.py`) a partir de uma descrição dos participantes e das mensagens. Isso garante que:

- os diagramas **não quebram** quando um texto muda — o layout é recalculado;
- estilo, cores e tamanhos ficam **consistentes** entre todos os diagramas do projeto;
- qualquer alteração de requisito pode ser refletida no diagrama com um único commit.

Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial com 6 diagramas de sequência

4.3 — Diagrama de Classes (UML)

Projeto: Fluxo — Sistema Bancário Digital **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09
Repositório: <https://github.com/AlfredoVentura/Fluxo>

0. Nota sobre o recorte deste diagrama

O item pedido em aula foi a **classe de assinatura**, tomando como exemplo um modelo de planos em que o cliente pode **retirar coisas** (itens, benefícios) daquilo que contratou. No Fluxo o mesmo raciocínio aparece em **dois** lugares, e por isso os dois foram modelados:

Diagrama	Onde está a "retirada"	O que modela
CLS-01	Retirada de itens da assinatura (plano/benefícios contratados)	Cliente, Plano, Assinatura, Item, ItemAssinatura, Cobranca, LogAuditoria
CLS-02	Retirada de dinheiro (saque) — que no Fluxo exige verificação em dois fatores	Usuario, VerificacaoDoisFatores, CanalVerificacao, DispositivoConfiavel, Conta, Retirada, LogAuditoria

Ou seja: o **caso da assinatura** (retirada de item) está no CLS-01 e o **caso do banco** (retirada de valor, com 2FA) está no CLS-02. Os dois compartilham o mesmo padrão de projeto: **nada é excluído fisicamente** — a retirada é registrada com data e motivo, preservando histórico e auditoria.

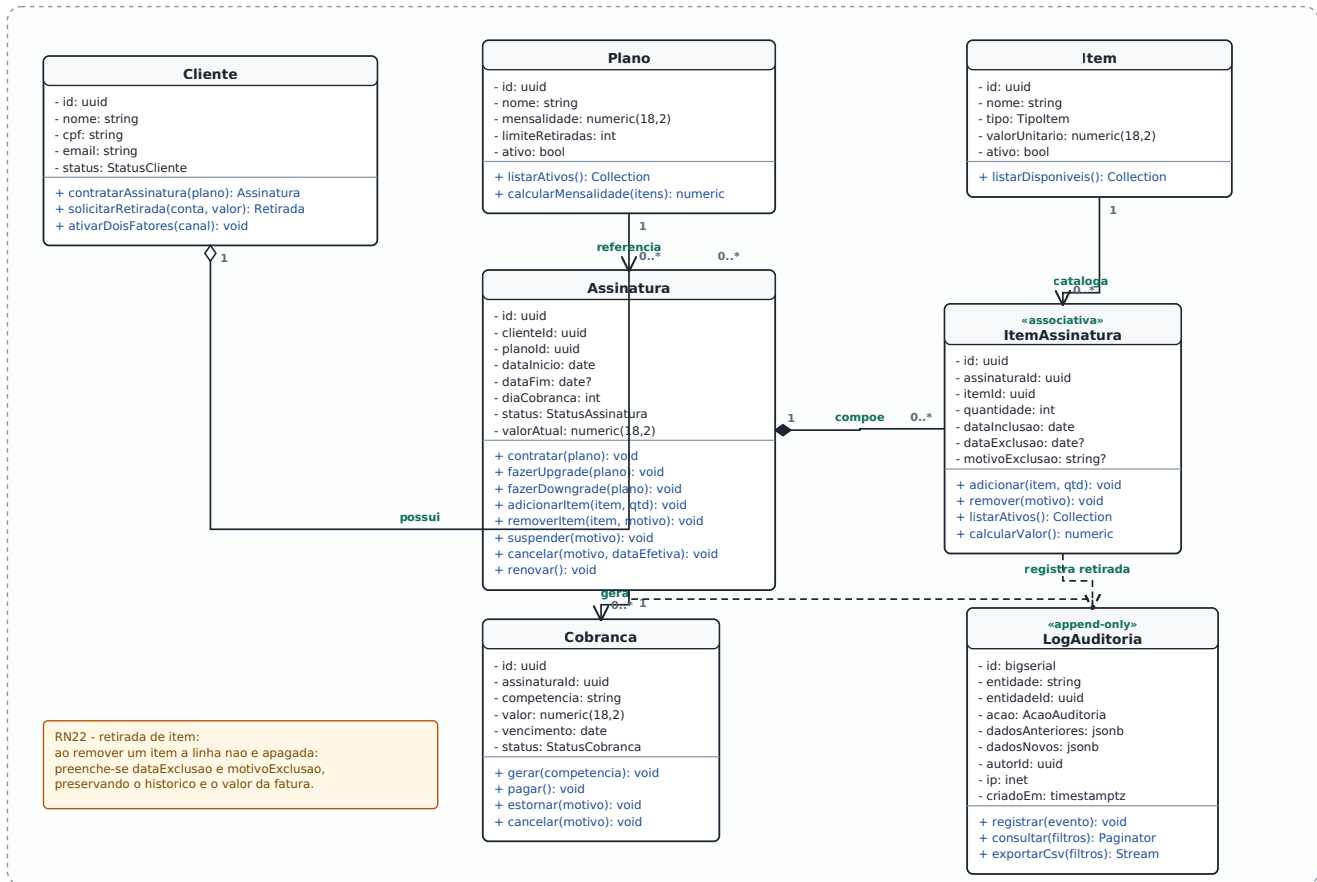
1. Convenções

Elemento	Notação
Classe	Retângulo com três compartimentos: estereótipo/nome, atributos, métodos
Atributo	- nome: tipo (privado)
Método	+ nome(param): retorno (público)
Associação	Linha cheia com seta aberta e multiplicidades nas pontas
Composição	Losango preenchido na ponta do todo (ciclo de vida dependente)
Agregação	Losango vazio na ponta do todo
Dependência	Linha tracejada com seta aberta («use»)
Estereótipos usados	«entity», «service», «enum», «associativa», «append-only»

2. CLS-01 — Assinatura, itens contratados e retirada de itens

CLS-01 — Classes de Assinatura, itens contratados e retirada de itens

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF65-RF72 · RN21-RN26



2.1 Descrição das classes

Cliente

Atributo	Tipo	Observação
id	uuid	Chave primária
nome	string	Nome completo
cpf	string	Único, validado por dígito verificador
email	string	Único, usado como login
status	StatusCliente	PENDENTE, ATIVO, BLOQUEADO, ENCERRADO

Métodos: contratarAssinatura(plano), solicitarRetirada(conta, valor), ativarDoisFatores(canal).

Plano

Catálogo de planos ofertados. Guarda mensalidade (valor base), limiteRetiradas (quantas retiradas o plano libera por mês sem tarifa) e o flag ativo.

Métodos: listarAtivos(), calcularMensalidade(itens).

Assinatura

Classe central do diagrama. É o vínculo entre um cliente e um plano, com vigência e regras de ciclo.

Atributo	Observação
dataInicio / dataFim	Vigência; dataFim nulo significa assinatura em curso
diaCobranca	Dia do mês em que a cobrança é gerada
status	ATIVA, SUSPensa, CANCELADA, PENDENTE
valorAtual	Mensalidade recalculada quando há inclusão ou retirada de item

Métodos relevantes

- fazerUpgrade(plano) / fazerDowngrade(plano) — troca de plano com efeito no próximo ciclo (RN21).
- adicionarItem(item, qtd) / removerItem(item, motivo) — delega para ItemAssinatura.
- suspender(motivo) — RN25 (atraso superior a 5 dias).
- cancelar(motivo, dataEfetiva) — RN24 (efeito ao fim do ciclo já pago).
- renovar() — cria o novo ciclo e a nova Cobrança.

Item

Catálogo de itens contratáveis (ex.: cartão adicional, saque excedente, relatório gerencial, conta remunerada). Atributo valorUnitario usado no cálculo proporcional.

ItemAssinatura — «associativa»

Classe associativa entre Assinatura e Item. É aqui que a **retirada de item** acontece:

Atributo	Observação
quantidade	Quantas unidades do item estão contratadas
dataInclusao	Data em que o item passou a valer
dataExclusao	Preenchido na retirada — nunca há DELETE (RN22)
motivoExclusao	Motivo informado pelo cliente ou pelo sistema

Métodos: adicionar(item, qtd), remover(motivo), listarAtivos() (filtra dataExclusao IS NULL), calcularValor().

Regra RN22 — retirada de item: ao remover um item, a linha **não é apagada**. Preenche-se dataExclusao e motivoExclusao, preservando o histórico, o valor da fatura do ciclo e a trilha de auditoria.

Cobrança

Ciclo de faturamento da assinatura (competencia no formato AAAA-MM), com valor, vencimento e status (ABERTA, PAGA, ATRASADA, CANCELADA, ESTORNADA).

Métodos: gerar(competencia), pagar(), estornar(motivo), cancelar(motivo).

LogAuditoria — «append-only»

Recebe um registro a cada inclusão, alteração ou **retirada** de item, troca de plano, cobrança gerada, suspensão e cancelamento. Detalhado em [LogAuditoria.md](#).

2.2 Multiplicidades

Relação	Significado
Cliente 1 – 0..* Assinatura	Um cliente pode ter nenhuma ou várias assinaturas ao longo do tempo
Plano 1 – 0..* Assinatura	Um plano é referenciado por muitas assinaturas
Assinatura 1 ◆– 0..* ItemAssinatura	Composição: os itens não existem sem a assinatura
Item 1 – 0..* ItemAssinatura	Um item do catálogo aparece em várias assinaturas
Assinatura 1 – 0..* Cobrança	Cada ciclo gera uma cobrança
ItemAssinatura ..> LogAuditoria	Dependência: a retirada de item registra auditoria

2.3 Esboço do modelo Laravel

```
<?php
// app/Models/Assinatura.php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\{BelongsTo, HasMany};

class Assinatura extends Model
{
    protected $fillable = ['cliente_id', 'plano_id', 'data_inicio', 'dia_cobranca', 'status'];

    protected $casts = [
        'data_inicio' => 'date',
        'data_fim' => 'date',
        'valor_atual' => 'decimal:2',
    ];

    public function cliente(): BelongsTo { return $this->belongsTo(Cliente::class); }
    public function plano(): BelongsTo { return $this->belongsTo(Plano::class); }
    public function itens(): HasMany { return $this->hasMany(ItemAssinatura::class); }
    public function cobrancas(): HasMany { return $this->hasMany(Cobranca::class); }

    /** Itens vigentes (sem data de exclusão). */
    public function itensAtivos(): HasMany
    {
        return $this->itens()->whereNull('data_exclusao');
    }

    /** RN22: a retirada NÃO apaga o registro – apenas fecha a vigência. */
    public function removerItem(Item $item, string $motivo): void
    {
        $vinculo = $this->itensAtivos()->where('item_id', $item->id)->firstOrFail();

        $vinculo->update([
            'data_exclusao' => now()->toDateString(),
            'motivo_exclusao' => $motivo,
        ]);

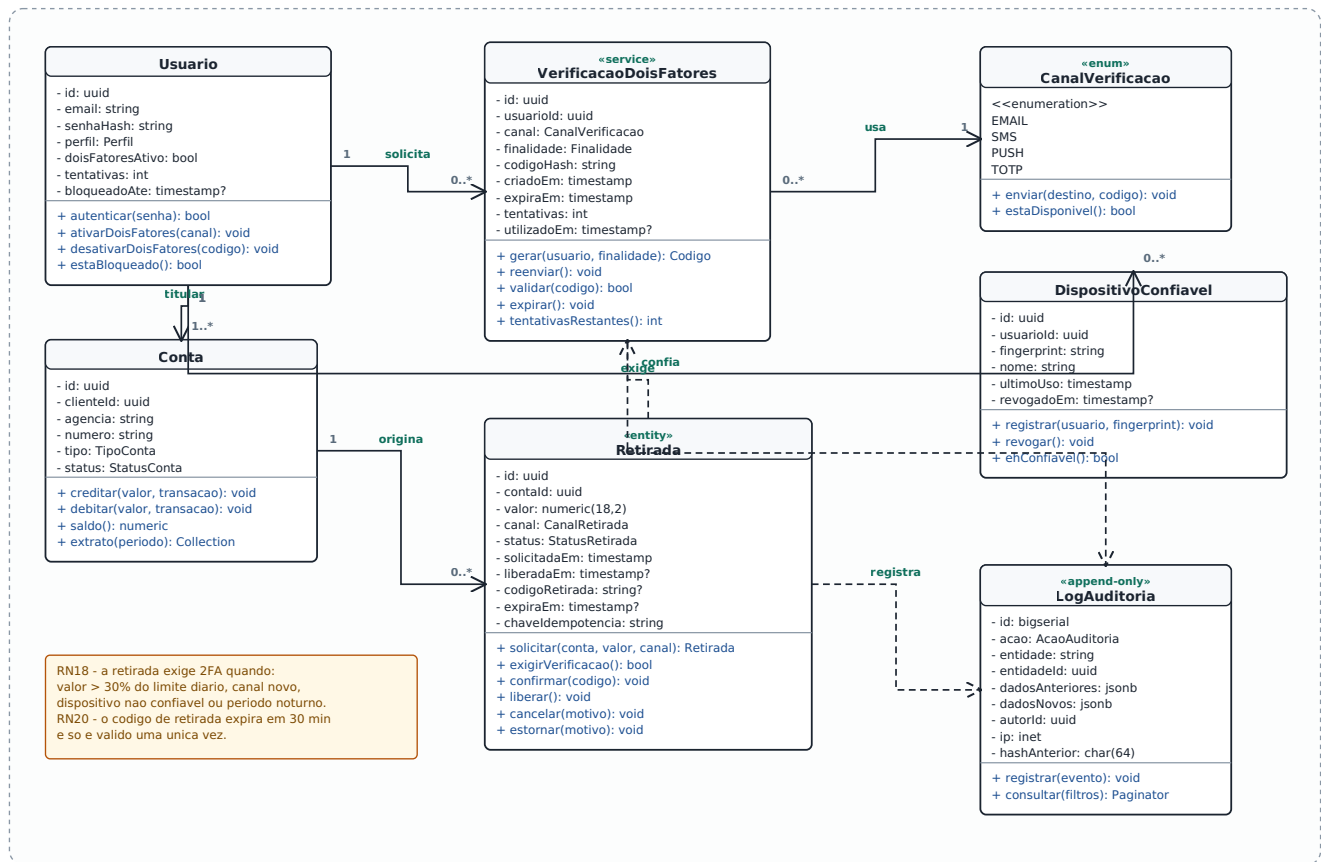
        $this->recalcularValor();
        $this->registrarAuditoria('ITEM_RETIRADO', $vinculo, $motivo);
    }

    /** RN23: valor proporcional aos itens ativos no dia da cobrança. */
    public function recalcularValor(): void
    {
        $itens = $this->itensAtivos()->sum('valor_total');
        $this->update(['valor_atual' => $this->plano->mensalidade + $itens]);
    }
}
```

3. CLS-02 — Retirada (saque) e Verificação em Dois Fatores

CLS-02 — Classes de Retirada (saque) e Verificação em Dois Fatores

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF53-RF64 · RN18-RN20



3.1 Descrição das classes

Usuario

Credenciais e política de acesso.

Atributo	Observação
senhaHash	bcrypt, custo 12 — nunca texto claro (RNF07)
perfil	CLIENTE, SUPORTE, BACKOFFICE, ADMINISTRADOR, AUDITOR
doisFatoresAtivo	Liga/desliga a exigência de 2FA
tentativas / bloqueadoAte	RF08 — 5 falhas em 15 min bloqueiam

Métodos: `autenticar(senha)`, `ativarDoisFatores(canal)`, `desativarDoisFatores(codigo)`, `estaBloqueado()`.

VerificacaoDoisFatores — «service»

Modela o **desafio** de verificação, não apenas o código.

Atributo	Observação
canal	EMAIL, SMS, PUSH, TOTP
finalidade	LOGIN, RETIRADA, ALTERACAO_SENSIVEL — impede reuso de código entre contextos

Atributo	Observação
codigoHash	SHA-256 do código; o código em si nunca é persistido (RF55)
criadoEm / expiraEm	5 min para login, 3 min para retirada (RN26)
tentativas	Máximo de 3 (RF56)
utilizadoEm	Uso único (RF54)

Métodos: gerar(usuario, finalidade), reenviar(), validar(codigo), expirar(), tentativasRestantes().

CanalVerificacao — «enum»

EMAIL, SMS, PUSH, TOTP. Método enviar(destino, codigo) e estaDisponivel().

DispositivoConfiavel

Permite dispensar o 2FA por 30 dias em dispositivos conhecidos (RF58). Guarda fingerprint, nome, ultimoUso e revogadoEm.

Conta

agencia, numero, tipo, status. **Não possui atributo de saldo** — o saldo é calculado pela soma dos lançamentos (RN02).

Métodos: creditar(valor, transacao), debitar(valor, transacao), saldo(), extrato(periodo).

Retirada — «entity»

O caso de uso de **retirada de dinheiro** do banco.

Atributo	Observação
valor	numeric(18,2) no banco, centavos na aplicação (RNF18)
canal	CAIXA_PARCEIRO, AGENCIA, TRANSFERENCIA
status	SOLICITADA, AGUARDANDO_2FA, LIBERADA, UTILIZADA, CANCELADA, EXPIRADA
codigoRetirada	Código de uso único entregue ao cliente (RN20)
expiraEm	30 minutos após a liberação
chaveIdempotencia	Impede duplicidade em reenvios (RNF17)

Métodos: solicitar(conta, valor, canal), exigirVerificacao(), confirmar(codigo), liberar(), cancelar(motivo), estornar(motivo).

LogAuditoria — «append-only»

Registra solicitação, desafio criado, falha de 2FA, liberação, utilização, cancelamento e estorno.

3.2 Regra de negócio central — RN18

```
public function exigirVerificacao(): bool
{
    $limiteDiario = $this->conta->limite_retirada_diaria; // RN19: padrão R$ 2.000,00
    $noturno      = now()->between('20:00', '06:00');    // RN19: cai para R$ 500,00
    $teto         = $noturno ? 500.00 : $limiteDiario;

    return $this->valor > ($teto * 0.30)                // acima de 30% do limite diário
        || $this->canalEhInedito()                      // canal nunca usado pelo cliente
        || ! $this->dispositivoEhConfiavel()            // dispositivo não confiável
        || $noturno;                                    // período noturno
}
```

3.3 Esboço do modelo Laravel

```
<?php
// app/Models/Retirada.php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Retirada extends Model
{
    public const STATUS = ['SOLICITADA', 'AGUARDANDO_2FA', 'LIBERADA',
                          'UTILIZADA', 'CANCELADA', 'EXPIRADA'];

    protected $casts = [
        'valor'      => 'decimal:2',
        'liberada_em' => 'datetime',
        'expira_em'  => 'datetime',
    ];

    public function conta() { return $this->belongsTo(Conta::class); }

    /** RN18 – condições que tornam o 2FA obrigatório. */
    public function exigirVerificacao(): bool { /* ...ver 3.2... */ }

    /** SEQ-04: só libera após o desafio validado e dentro de transação ACID. */
    public function confirmar(string $codigo): void
    {
        $desafio = VerificacaoDoisFatores::where('retirada_id', $this->id)
            ->where('finalidade', 'RETIRADA')
            ->whereNull('utilizado_em')
            ->firstOrFail();

        if (! $desafio->validar($codigo)) {
            $this->registrarAuditoria('RETIRADA_2FA_FALHOU');
            throw new CodigoInvalidoException('Código inválido, expirado ou já utilizado.');
        }

        \DB::transaction(function () {
            $transacao = Transacao::criar($this->conta, $this->valor, 'RETIRADA');
            $this->conta->debitar($this->valor, $transacao); // RN04: sem saldo negativo
            Conta::caixa()->creditar($this->valor, $transacao);

            $this->update([
                'status'      => 'LIBERADA',
                'liberada_em' => now(),
                'codigo_retirada' => str_pad((string) random_int(0, 999999), 6, '0', STR_PAD_LEFT),
                'expira_em'   => now()->addMinutes(30), // RN20
            ]);

            $this->registrarAuditoria('RETIRADA_LIBERADA');
        });
    }
}
```

4. Mapeamento classes → entidades do banco de dados

Classe	Tabela (PostgreSQL)	Observação
Cliente	clientes	1:1 com usuarios
Usuario	usuarios	Credenciais, perfil e 2FA
Conta	contas	Sem coluna de saldo
Retirada	retiradas	FK para contas e transacoes
VerificacaoDoisFatores	codigos_verificacao	Hash + expiração + tentativas
DispositivoConfiavel	dispositivos_confiaveis	0..* por usuário
Plano	planos	Catálogo
Assinatura	assinaturas	FK cliente_id, plano_id
Item	itens	Catálogo
ItemAssinatura	itens_assinatura	Classe associativa; sem exclusão física
Cobranca	cobrancas	Uma por competência
LogAuditoria	logs_auditoria	Append-only, com trigger de proteção

Com estas 12 classes, o modelo passa das 16 entidades já previstas para **21 entidades**, superando com folga o mínimo de 8 exigido pela disciplina.

5. Rastreabilidade

Requisito	Classe / diagrama
RF53-RF58 (2FA)	VerificacaoDoisFatores, CanalVerificacao, DispositivoConfiavel (CLS-02)
RF59-RF64 (retirada)	Retirada, Conta (CLS-02)
RF65-RF72 (assinatura)	Assinatura, Plano, Item, ItemAssinatura, Cobranca (CLS-01)
RF45, RF46, RF48 (auditoria)	LogAuditoria (CLS-01 e CLS-02)
RN18, RN19, RN20	Retirada::exigirVerificacao(), Retirada::confirmar()
RN21-RN25	Assinatura, ItemAssinatura, Cobranca
RN26	VerificacaoDoisFatores
RN27	LogAuditoria
RNF07, RNF18	Usuario::\$senhaHash, valores numeric(18,2)

Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial: CLS-01 (assinatura e retirada de itens) e CLS-02 (retirada/saque com 2FA)

4.3 — Diagramas de Atividades (UML)

Projeto: Fluxo — Sistema Bancário Digital **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09
Repositório: <https://github.com/AlfredoVentura/Fluxo>

1. Convenções

Elemento	Notação
Nó inicial	Círculo preenchido
Ação	Retângulo de cantos arredondados
Decisão	Losango, com o enunciado acima e as guardas entre colchetes ([sim] , [nao])
Nó final	Círculo com anel interno preenchido
Raia (swimlane)	Coluna com o nome do responsável pela ação
Fluxo de retorno	Linha contornando a lateral (realimentação)

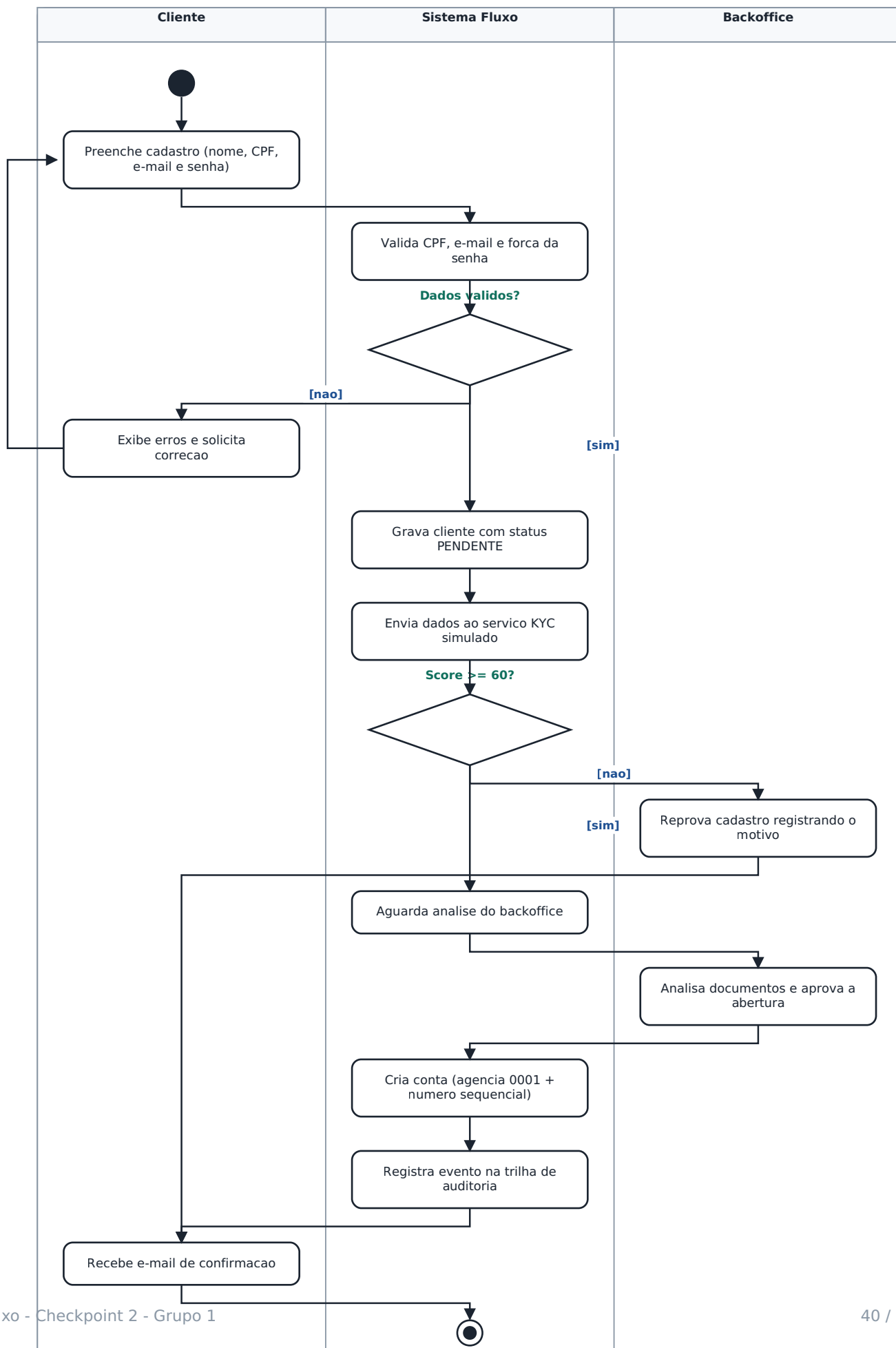
Foram modelados **5 diagramas**, um para cada fluxo de maior risco ou maior complexidade de regra.

ID	Diagrama	Raias	Requisitos
ATV-01	Abertura de conta com KYC	Cliente · Sistema Fluxo · Backoffice	RF01–RF05, RN01
ATV-02	Transferência Pix com análise de risco	Cliente · Sistema Fluxo · Motor Antifraude	RF18–RF21, RN02–RN09
ATV-03	Retirada (saque) com 2FA	Cliente · Sistema Fluxo · Serviço 2FA · Conta/ Caixa	RF59–RF64, RN18–RN20
ATV-04	Pagamento de boleto	Cliente · Sistema Fluxo · Registradora	RF36, RF37, RF25
ATV-05	Ciclo da assinatura e retirada de itens	Cliente · Sistema Fluxo · Cobrança	RF65–RF72, RN21–RN25

2. ATV-01 — Abertura de conta com verificação de identidade (KYC)

ATV-01 — Atividade: abertura de conta com verificação de identidade (KYC)

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF01-RF05 · RN01



Decisões

Decisão	Guarda	Caminho
Dados válidos?	[nao]	O sistema exibe os erros e devolve o cliente ao formulário (realimentação)
	[sim]	Grava o cliente com <code>status = PENDENTE</code> e envia ao KYC
Score $\geq$ 60?	[nao]	Reprova o cadastro registrando o motivo e notifica o cliente
	[sim]	Aguarda a análise humana do backoffice

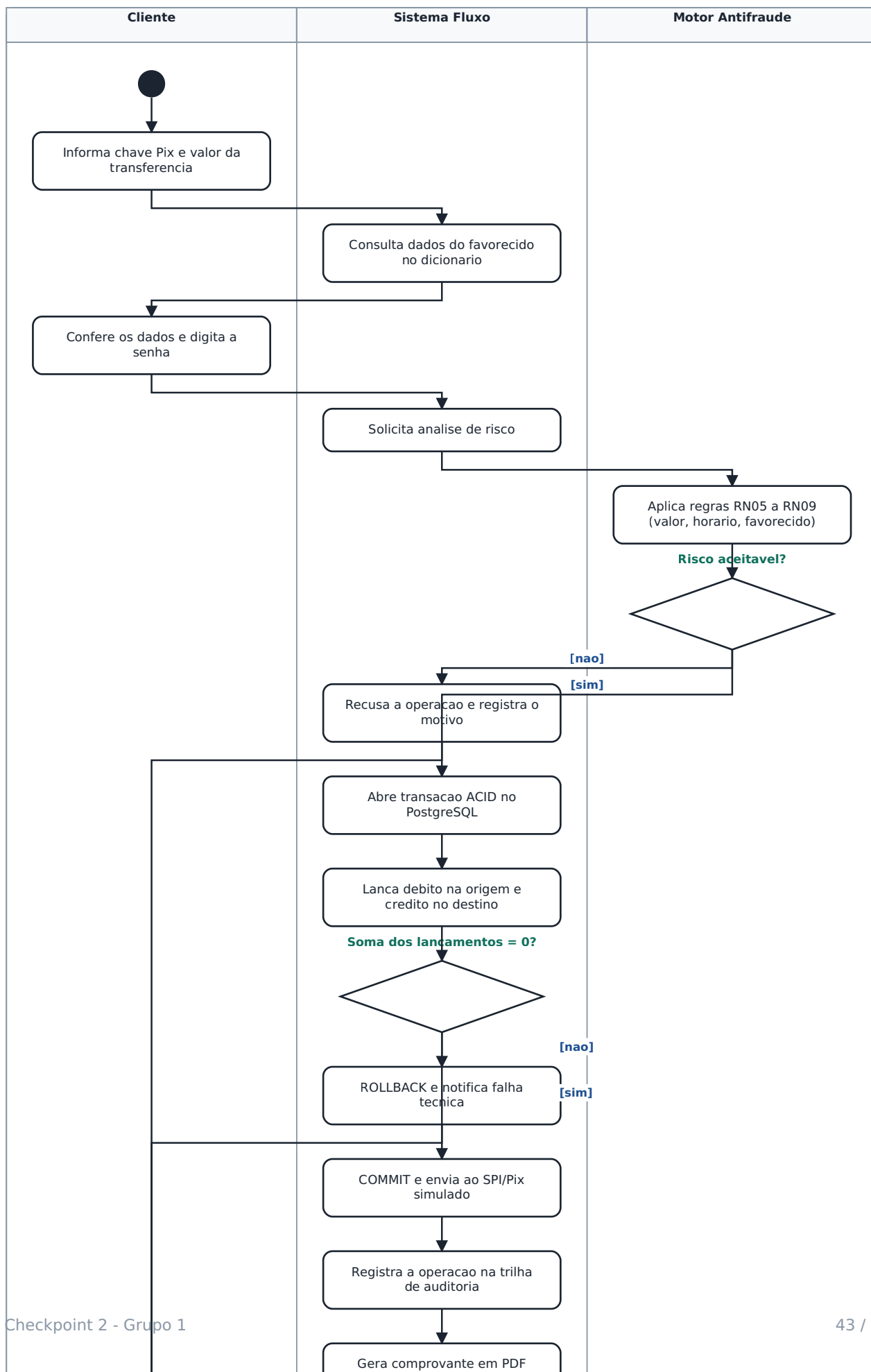
Regras exercitadas

- **RN01** — a conta só é ativada após aprovação do backoffice.
 - **RF02** — dígito verificador do CPF e unicidade conferidos antes de gravar.
 - **RNF07** — senha com hash bcrypt, custo 12.
-

3. ATV-02 — Transferência Pix com análise de risco e partidas dobradas

ATV-02 — Atividade: transferência Pix com análise de risco e partidas dobradas

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF18-RF21 · RN02-RN09 · RNF15-RNF17



Decisões

Decisão	Guarda	Caminho
Risco aceitável?	[nao]	Recusa a operação, registra o motivo e notifica o cliente
	[sim]	Abre transação ACID e lança débito/crédito
Soma dos lançamentos = 0?	[nao]	ROLLBACK e notificação de falha técnica
	[sim]	COMMIT , envio ao SPI/Pix simulado, auditoria, comprovante e notificação

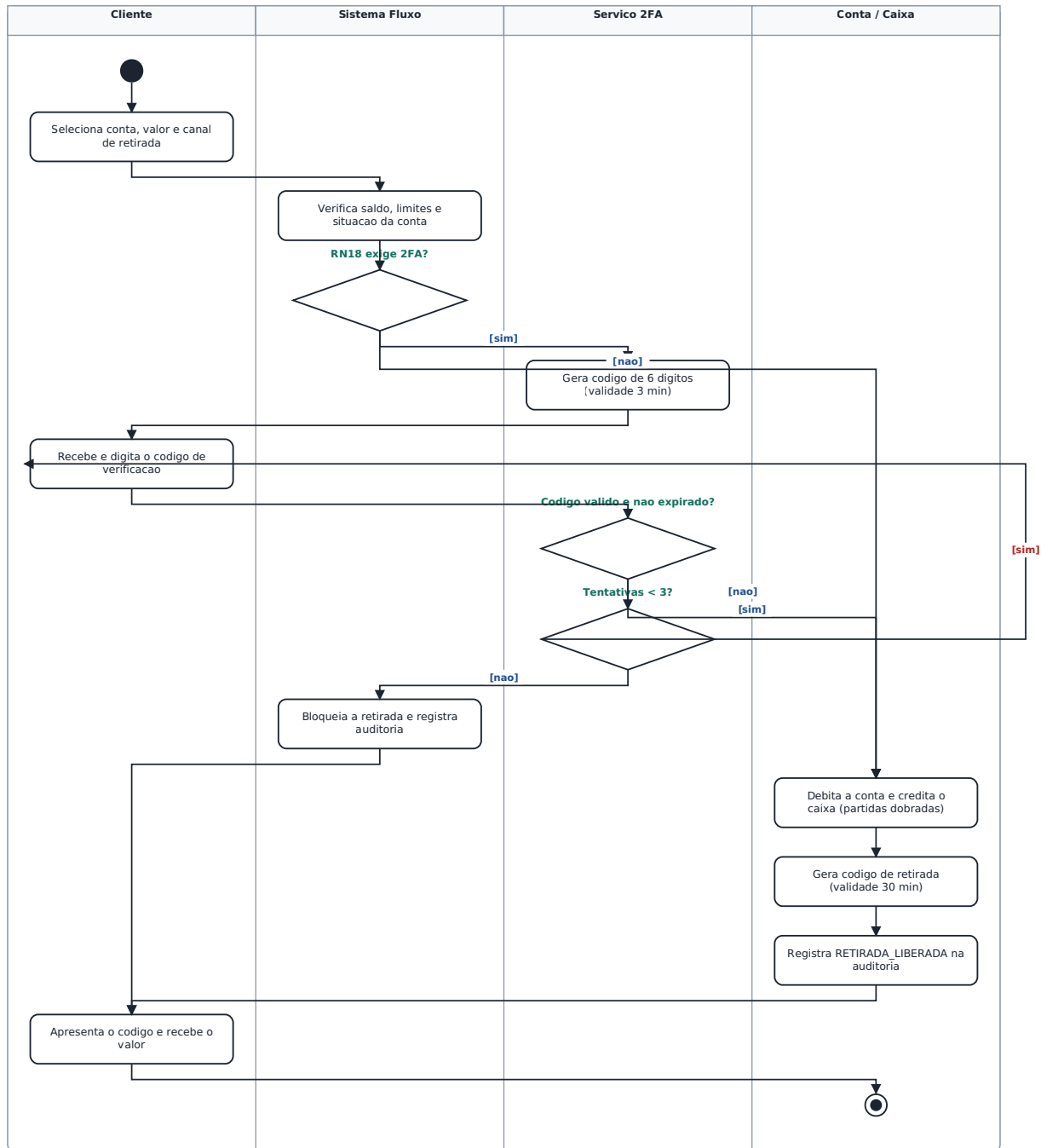
Regras exercitadas

- **RN02 / RN03** — saldo é derivado dos lançamentos; toda transação tem soma zero.
 - **RN05-RN09** — valor por operação, horário noturno, limite diário, favorecido novo e favorecido cadastrado há menos de 24 h.
 - **RNF15-RNF17** — transação ACID, verificação de consistência e idempotência.
-

4. ATV-03 — Retirada (saque) com verificação em dois fatores

ATV-03 — Atividade: retirada (saque) com verificação em dois fatores

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF59-RF64 · RN18-RN20 · RF45



Decisões

Decisão	Guarda	Caminho
RN18 exige 2FA?	[nao]	Vai direto para o débito (valores baixos, canal conhecido, dispositivo confiável, horário comercial)
	[sim]	Gera e envia o código de 6 dígitos
Código válido e não expirado?	[nao]	Verifica as tentativas
	[sim]	Debita a conta, credita o caixa e gera o código de retirada

Decisão	Guarda	Caminho
Tentativas < 3?	[sim]	Solicita novo código (realimentação)
	[nao]	Bloqueia a retirada e registra RETIRADA_2FA_FALHOU

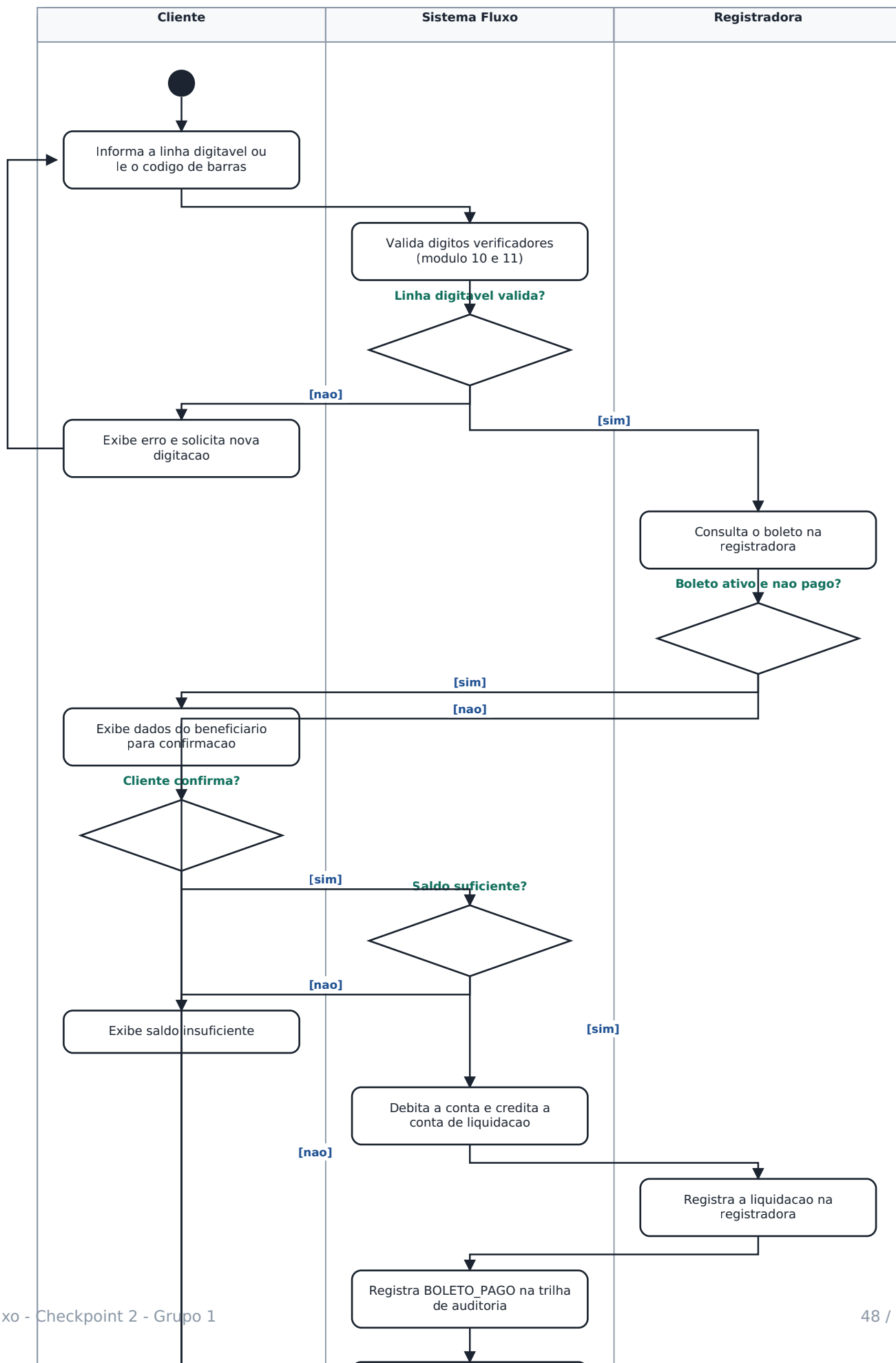
Regras exercitadas

- **RN18** — condições que tornam o 2FA obrigatório (30% do limite diário, canal inédito, dispositivo não confiável, período noturno).
- **RN19** — limite diário de R\$ 2.000,00 (R\$ 500,00 entre 20h e 6h).
- **RN20** — código de retirada de uso único, válido por 30 minutos; ao expirar, estorno automático.
- **RN26** — código de verificação com uso único, 3 minutos para retirada, no máximo 3 tentativas.

5. ATV-04 — Pagamento de boleto

ATV-04 — Atividade: pagamento de boleto com validação e liquidação simulada

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF36, RF37, RF25 · RN04, RN10



Decisões

Decisão	Guarda	Caminho
Linha digitável válida?	[nao]	Exibe o erro e solicita nova digitação (realimentação)
	[sim]	Consulta o boleto na registradora
Boleto ativo e não pago?	[nao]	Exibe a situação ao cliente e encerra
	[sim]	Mostra beneficiário, valor e vencimento para confirmação
Cliente confirma?	[nao]	Encerra sem cobrança
	[sim]	Verifica o saldo
Saldo suficiente?	[nao]	Exibe "saldo insuficiente" e encerra (RN04 — não há cheque especial)
	[sim]	Debita a conta, credita a conta de liquidação e registra a liquidação

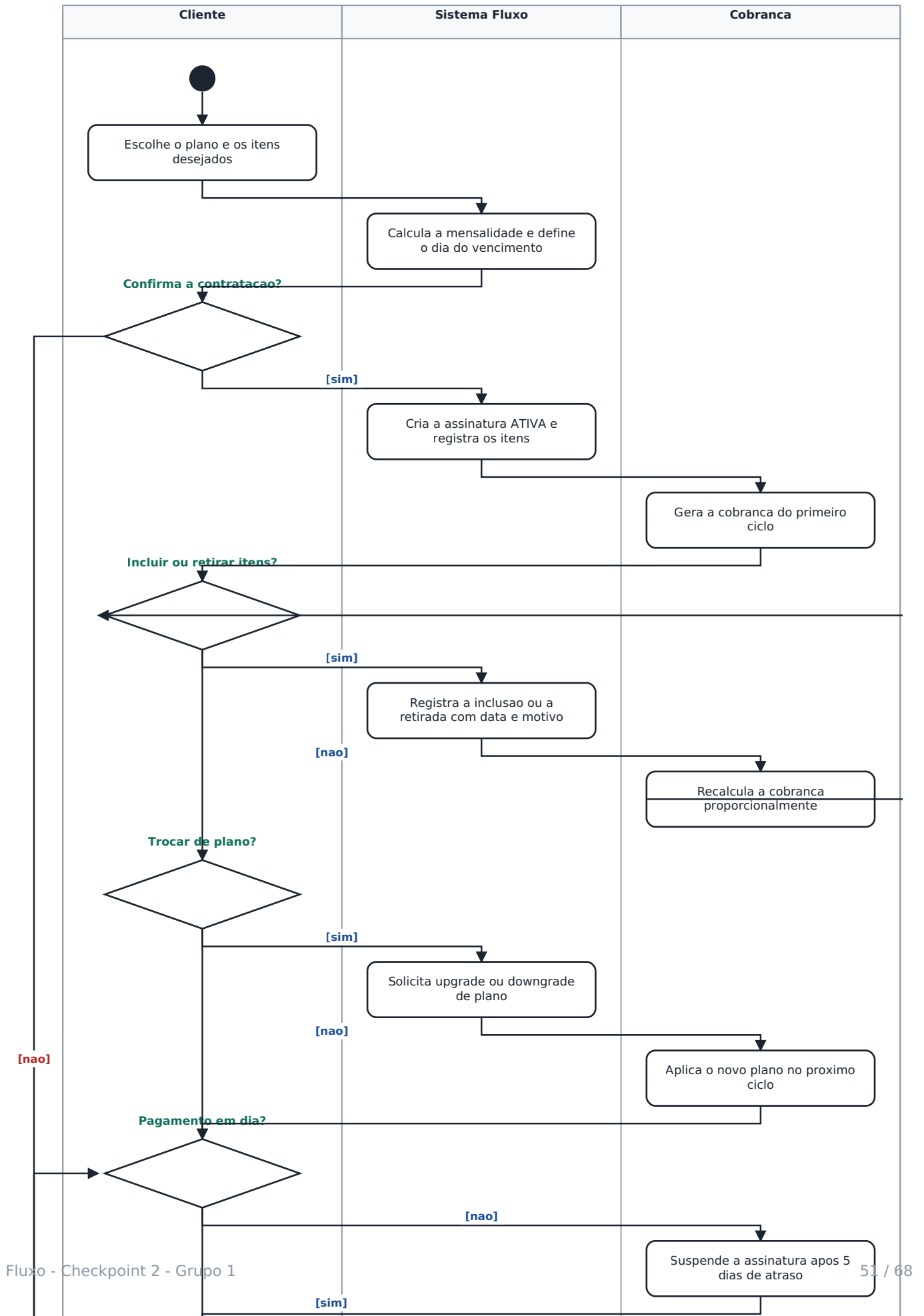
Regras exercitadas

- **RF37** — validação dos dígitos verificadores (módulo 10 e módulo 11) antes de qualquer consulta externa.
 - **RN04** — não é permitido saldo negativo.
 - **RN10** — transações efetivadas não são excluídas; correções ocorrem por estorno.
-

6. ATV-05 — Ciclo de vida da assinatura, inclusão e retirada de itens

ATV-05 — Atividade: ciclo de vida da assinatura, inclusão e retirada de itens

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF65-RF72 · RN21-RN25



Decisões

Decisão	Guarda	Caminho
Confirma a contratação?	[nao]	Encerra sem gerar cobrança
	[sim]	Cria a assinatura ATIVA e registra os itens
Incluir ou retirar itens?	[sim]	Registra a movimentação com data e motivo e recalcula a cobrança (realimentação)
	[nao]	Segue para a troca de plano
Trocar de plano?	[sim]	Solicita upgrade/downgrade, com efeito no próximo ciclo
	[nao]	Segue para a verificação de pagamento
Pagamento em dia?	[nao]	Suspende após 5 dias de atraso; o cliente regulariza e o ciclo é reavaliado
	[sim]	Segue para o cancelamento/renovação
Cancelar a assinatura?	[sim]	Cancela com efeito no fim do ciclo pago e registra na trilha
	[nao]	Renova a assinatura no vencimento

Regras exercitadas — o coração do item "retirada de coisas"

- **RN22** — a **retirada de item** não apaga o registro: preenche `dataExclusao` e `motivoExclusao`, preservando o histórico e o valor da fatura do ciclo.
- **RN23** — a cobrança é gerada com base nos itens ativos na data do vencimento; inclusões e retiradas no meio do ciclo geram cobrança **proporcional**.
- **RN21** — só existe um plano ativo por vez; a troca cria nova vigência preservando a anterior.
- **RN24** — o cancelamento vale a partir do fim do ciclo já pago (sem reembolso proporcional nesta versão).
- **RN25** — atraso superior a 5 dias suspende a assinatura e bloqueia os itens.

7. Rastreabilidade atividade × requisito

Requisito	ATV-01	ATV-02	ATV-03	ATV-04	ATV-05
RF01-RF05 (cadastro/KYC)	☐				
RF18-RF21 (Pix)		☐			
RF25 (comprovante PDF)		☐		☐	
RF36, RF37 (boleto)				☐	
RF53-RF58 (2FA)			☐		
RF59-RF64 (retirada)			☐		
RF65-RF72 (assinatura)					☐
RN01	☐				
RN02, RN03 (partidas dobradas)		☐	☐	☐	
RN04 (sem saldo negativo)		☐	☐	☐	
RN05-RN09 (risco)		☐			
RN18-RN20 (retirada)			☐		
RN21-RN25 (assinatura)					☐

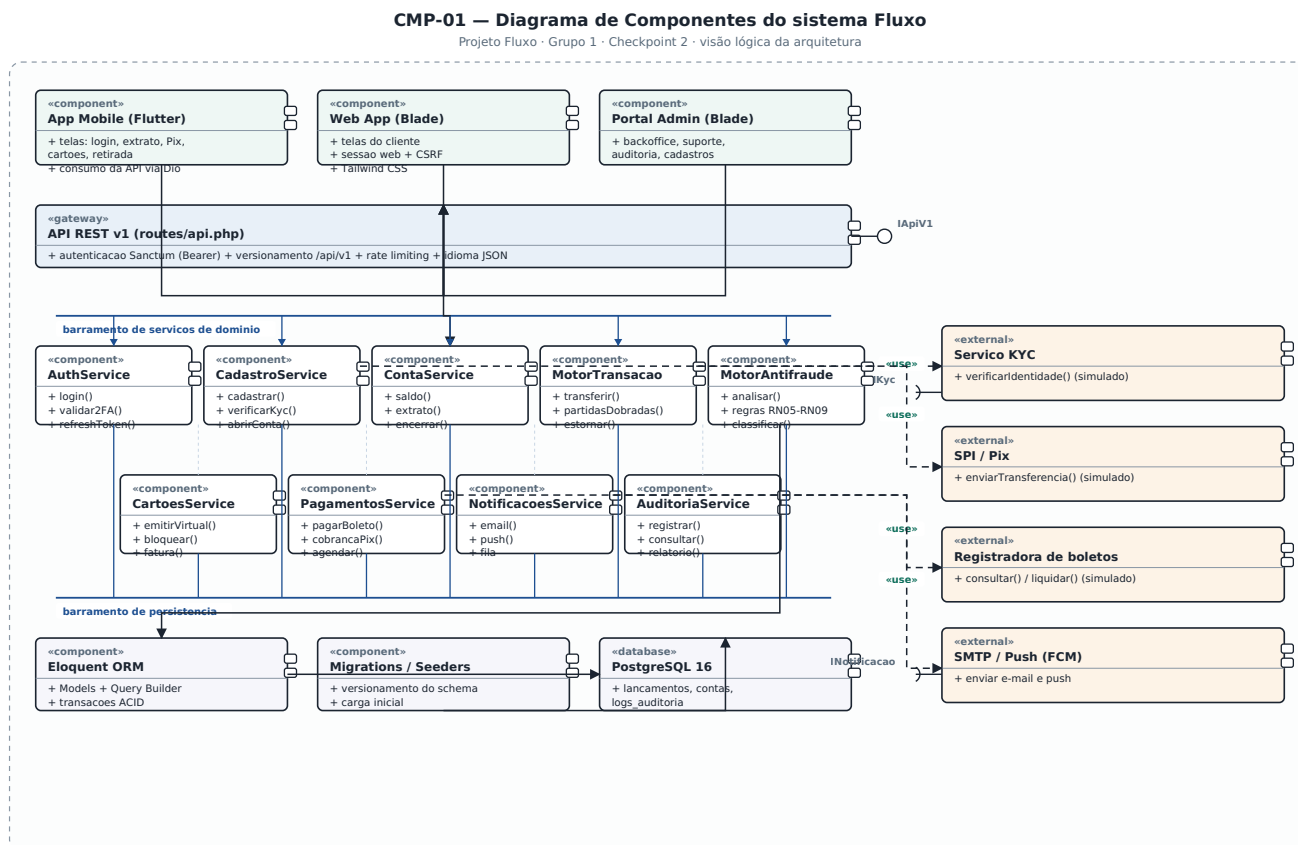
Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial com 5 diagramas de atividades

4.3 — Diagramas de Componentes e de Implantação (UML)

Projeto: Fluxo — Sistema Bancário Digital **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09
Repositório: <https://github.com/AlfredoVentura/Fluxo>

1. CMP-01 — Diagrama de Componentes (visão lógica)



1.1 Camadas

Camada	Componentes	Responsabilidade
Apresentação	App Mobile (Flutter), Web App (Blade), Portal Admin (Blade)	Interface com o usuário; o app consome a API via HTTPS/JSON, a web usa sessão + CSRF
Fronteira	API REST v1 («gateway»)	Autenticação Sanctum, versionamento /api/v1 , rate limiting, validação de requisição
Domínio	9 serviços de negócio	Regras do banco; cada serviço é um componente substituível
Persistência	Eloquent ORM, Migrations/Seeders, PostgreSQL 16	Mapeamento objeto-relacional, versionamento do esquema e armazenamento
Externa	Serviço KYC, SPI/Pix, Registradora de boletos, SMTP/Push	Integrações simuladas — nenhuma opera dinheiro real

1.2 Componentes de domínio

Componente	Interfaces principais	Requisitos
AuthService	login(), validar2FA(), refreshToken()	RF06-RF08, RF53-RF58
CadastroService	cadastrar(), verificarKyc(), abrirConta()	RF01-RF05
ContaService	saldo(), extrato(), encerrar()	RF13-RF15
MotorTransacao	transferir(), partidasDobradas(), estornar()	RF18-RF24, RN02, RN03
MotorAntifraude	analisar(), regras RN05-RN09, classificar()	RF20, RN05-RN09
CartoesService	emitirVirtual(), bloquear(), fatura()	RF27-RF35
PagamentosService	pagarBoleto(), cobrancaPix(), agendar()	RF36-RF42
NotificacoesService	email(), push(), fila	RF42
AuditoriaService	registrar(), consultar(), relatorio()	RF45, RF46, RF48

1.3 Interfaces oferecidas e requeridas

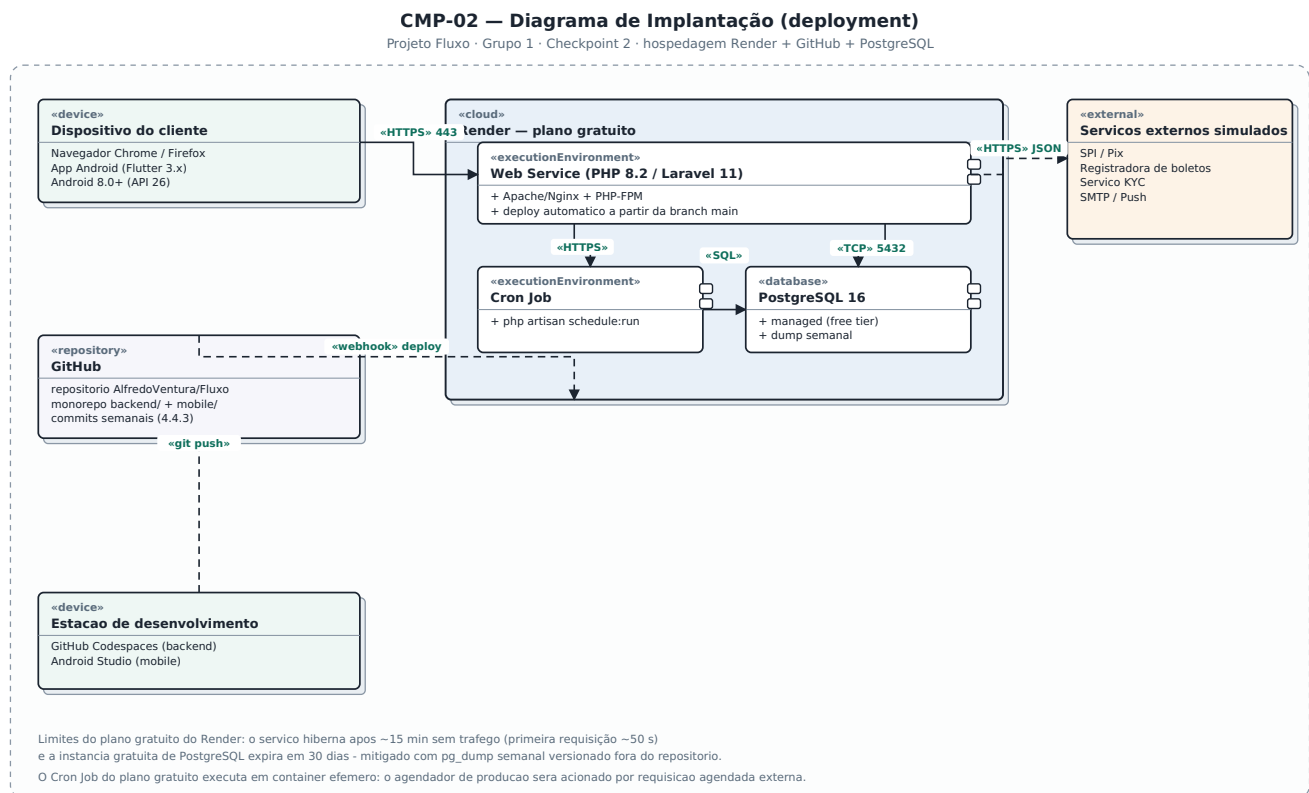
Notação	Interface	De	Para
Bola (I)	IApiV1	API REST v1	App Mobile, Web App, Portal Admin
Soquete (C)	IKyc	CadastroService	Serviço KYC
Soquete (C)	INotificacao	NotificacoesService	SMTP / Push

CadastroService, MotorTransacao, PagamentosService e NotificacoesService dependem de componentes externos por relacionamentos «use» tracejados — a troca do simulador por um serviço real não altera o contrato interno.

1.4 Decisões arquiteturais

- Uma API para dois clientes.** App mobile e web consomem o mesmo contrato `/api/v1`; a web usa adicionalmente rotas Blade com sessão e CSRF. Isso evita duplicidade de regra de negócio (RNF30).
- Serviços de domínio isolados dos controllers.** O controller apenas valida e delega; a regra vive no serviço, o que viabiliza os testes automatizados exigidos por RNF27.
- Auditoria como componente transversal.** Todo serviço de domínio depende de AuditoriaService, garantindo RF45 sem duplicar código.
- Persistência atrás do ORM.** Nenhum serviço escreve SQL direto; migrations versionam o esquema (RNF28).
- Notificações por fila.** `email()` e `push()` são assíncronos, preservando RNF02 (resposta de escrita em até 3 s).

2. CMP-02 — Diagrama de Implantação (deployment)



2.1 Nós

Nó	Estereótipo	Conteúdo
Dispositivo do cliente	«device»	Navegador Chrome/Firefox e app Android (Flutter 3.x, Android 8.0+ / API 26)
Estação de desenvolvimento	«device»	GitHub Codespaces (backend) e Android Studio (mobile)
GitHub	«repository»	Repositório AlfredoVentura/Fluxo , monorepo backend/ + mobile/
Render	«cloud»	Web Service PHP 8.2/Laravel 11, Cron Job, PostgreSQL 16 gerenciado
Serviços externos simulados	«external»	SPI/Pix, registradora de boletos, serviço KYC, SMTP/Push

2.2 Conexões e protocolos

De	Para	Protocolo	Observação
Dispositivo do cliente	Web Service	«HTTPS» 443	TLS 1.2+ obrigatório (RNF06)
Web Service	PostgreSQL	«TCP» 5432	Conexão interna do datacenter do Render
Web Service	Cron Job	«HTTPS»	O agendador é acionado por requisição, já que o Cron Job gratuito é efêmero
Cron Job	PostgreSQL	«SQL»	Execução de schedule:run e expiração de retiradas/cobranças
GitHub	Render	«webhook» deploy	Cada push em main dispara build e deploy automáticos

De	Para	Protocolo	Observação
Estação de desenvolvimento	GitHub	«git push»	Commits semanais (item 4.4.3)
Web Service	Serviços externos	«HTTPS» JSON	Integrações simuladas

2.3 Limites do plano gratuito do Render — e como o grupo contorna cada um

Limitação	Impacto	Mitigação adotada
O serviço hiberna após ~15 min sem tráfego	Primeira requisição leva ~50 s	"Aquecer" a URL antes da apresentação; manter vídeo de contingência da demonstração
A instância gratuita de PostgreSQL expira em 30 dias	Perda do banco	pg_dump semanal versionado fora do repositório; migrations + seeders como fonte de verdade para recriar o banco (RNF19)
Cron Job roda em container efêmero	Não mantém estado entre execuções	O agendador de produção é acionado por requisição HTTP agendada externamente; nenhum estado é guardado no container
Sem disco persistente	Uploads se perdem	Anexos de chamado são gravados no banco (bytea) ou em storage externo gratuito
Build com limite de tempo	Build de Flutter fora do Render	O APK é gerado no Android Studio e publicado como release no GitHub

3. Organização do repositório (monorepo)

```

Fluxo/
├── backend/                                # Laravel 11 (PHP 8.2+)
│   ├── app/
│   │   ├── Http/Controllers/              # AuthController, TransacaoController, RetiradaController...
│   │   ├── Models/                       # Cliente, Conta, Retirada, Assinatura, LogAuditoria...
│   │   ├── Services/                     # MotorTransacao, MotorAntifraude, AuditoriaService...
│   │   └── Policies/                     # Controle de acesso por perfil (RF44)
│   ├── database/migrations/              # Versionamento do esquema (RNF28)
│   ├── resources/views/                  # Telas Blade + Tailwind
│   ├── routes/api.php                    # API REST v1
│   └── tests/                            # PHPUnit – meta de 60% de cobertura (RNF27)
├── mobile/                               # Flutter (Dart)
│   ├── lib/screens/                      # Login, Dashboard, Extrato, Pix, Retirada, Assinatura
│   ├── lib/services/                     # Cliente HTTP da API
│   └── test/                             # Testes unitários e de widget

```

4. Rastreabilidade

Requisito	Evidência no diagrama
RNF06 (HTTPS)	CMP-02 — «HTTPS» em todas as conexões externas
RNF31 (deploy automático a partir do GitHub)	CMP-02 — webhook GitHub → Render
RNF33–RNF37 (restrições de stack)	CMP-01 (Laravel, Blade, Flutter) e CMP-02 (Render, PostgreSQL)
RNF38 (acesso do professor ao repositório)	CMP-02 — nó GitHub, repositório público
RF45 (auditoria transversal)	CMP-01 — AuditoriaService dependido por todo o domínio
RNF19 (backup semanal)	CMP-02 — nota de mitigação do free tier

Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial: CMP-01 (componentes) e CMP-02 (implantação)

Log de Auditoria (Trilha de Auditoria Imutável)

Projeto: Fluxo — Sistema Bancário Digital **Grupo:** 1 · **Checkpoint 2** — apresentação em 17/09
Repositório: <https://github.com/AlfredoVentura/Fluxo> **Versão:** 1.0

1. Objetivo

Registrar, de forma **imutável, completa e verificável**, todo evento relevante ocorrido no Fluxo — quem fez, o que fez, quando, de onde, sobre qual entidade e qual era o estado antes e depois — de modo que seja possível:

1. **Reconstruir** a história de qualquer conta, transação, assinatura ou usuário;
2. **Detectar** adulteração (hash encadeado);
3. **Atender** o auditor e o administrador com consultas filtradas e exportação;
4. **Provar conformidade** com a LGPD (finalidade, minimização, rastreabilidade de acesso a dados pessoais).

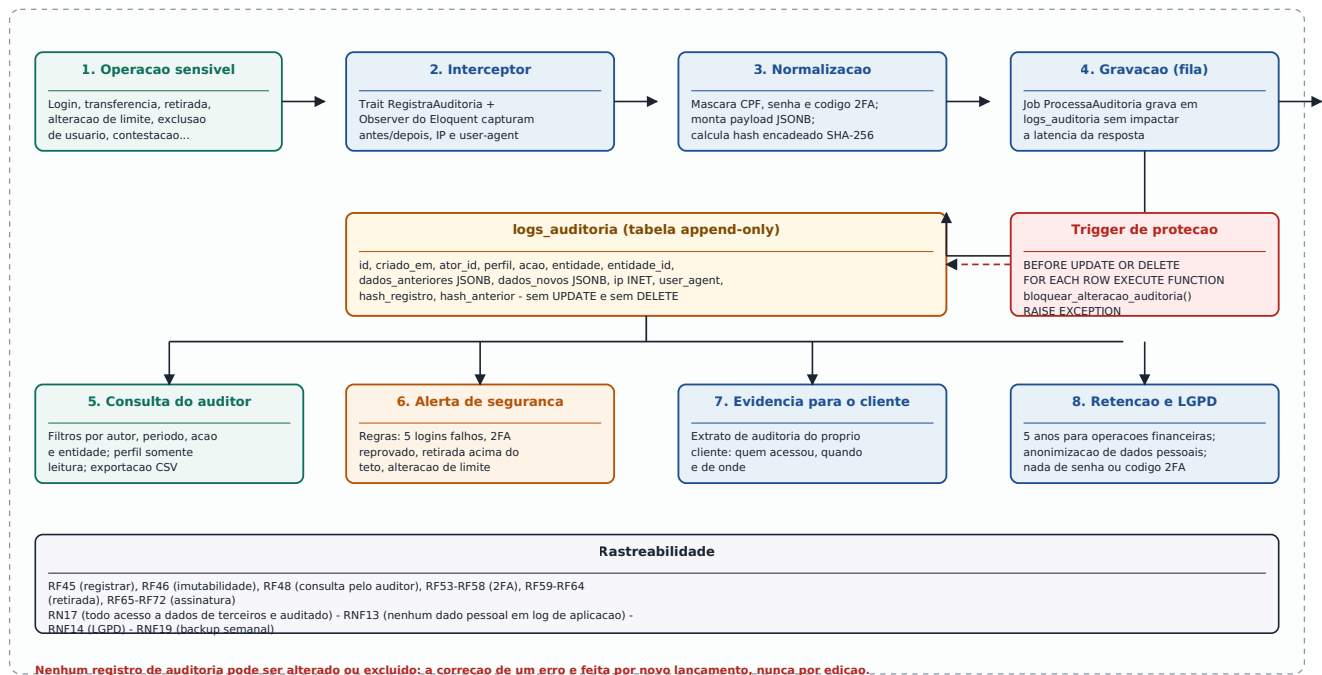
Requisitos atendidos

Requisito	Descrição
RF45	Registrar em trilha de auditoria toda operação sensível, com autor, data/hora, IP e dados alterados
RF46	Impedir a alteração ou exclusão de registros da trilha
RF48	Permitir ao auditor consultar a trilha com filtros por autor, período e tipo de operação
RF73	Registrar toda tentativa de autenticação, bem-sucedida ou não, com IP e dispositivo
RF74	Registrar inclusão, alteração e retirada de itens da assinatura
RF75	Permitir ao auditor exportar o resultado da consulta em CSV
RF76	Encadear os registros por hash SHA-256, permitindo detectar adulteração
RN17	Todo acesso ou alteração a dados de terceiros por usuário interno é registrado
RN27	Nenhum registro da trilha pode ser alterado ou excluído
RNF13	Nenhum dado pessoal em log de aplicação
RNF41	A gravação da trilha não acrescenta mais de 100 ms à resposta
RNF42	Deteção de adulteração por verificação diária do hash encadeado
RNF43	Nenhum dado sensível (senha, código 2FA, CVV, token) no payload
RNF44	Retenção de 5 anos

2. Fluxo do log de auditoria

Log de Auditoria — fluxo de registro, proteção e consulta

Projeto Fluxo · Grupo 1 · Checkpoint 2 · RF45, RF46, RF48 · RN17



Etapas	Onde acontece	Observação
1. Operação sensível	Controllers e serviços de domínio	Listada na seção 3
2. Interceptor	Trait RegistraAuditoria + Observer do Eloquent	Captura antes e depois automaticamente
3. Normalização	AuditoriaPayload	Mascara dados sensíveis, monta JSONB, calcula hash
4. Gravação	Job ProcessaAuditoria (fila)	Fora da requisição — não impacta a latência (RNF41)
5-8. Consumo	Consulta do auditor, alertas, evidência do cliente, retenção	Somente leitura + exportação

3. Matriz de eventos auditáveis

Legenda de nível: INFO (informativo) · ATENCAO (merece revisão) · CRITICO (exige ação).

3.1 Acesso e segurança

Ação	Nível	Quando	Requisito
LOGIN_SUCESSO	INFO	Autenticação concluída	RF73
LOGIN_FALHOU	ATENCAO	Senha incorreta ou conta bloqueada	RF08, RF73
LOGIN_2FA_SUCESSO	INFO	Código validado	RF54
2FA_FALHOU	ATENCAO	Código inválido, expirado ou reutilizado	RF56
2FA_ATIVADO / 2FA_DESATIVADO	ATENCAO	Cliente liga/desliga o segundo fator	RF53
LOGOUT	INFO	Encerramento de sessão	—

Ação	Nível	Quando	Requisito
SENHA_ALTERADA / SENHA_RECUPERADA	ATENCAO	Troca e recuperação de senha	RF09
DISPOSITIVO_CONFIAVEL_REVOGADO	ATENCAO	Revogação de dispositivo	RF58

3.2 Conta e cadastro

Ação	Nível	Quando
CADASTRO_SOLICITADO	INFO	Cliente submete o cadastro (RF01)
CADASTRO_APROVADO / CADASTRO_REPROVADO	ATENCAO	Decisão do backoffice (RF04)
CONTA_ABERTA / CONTA_ENCERRADA	ATENCAO	Criação e encerramento (RF05, RF12)
CONTA_BLOQUEADA / CONTA_DESBLOQUEADA	CRITICO	Bloqueio por suspeita de fraude (RF47)
DADOS_CADASTRAS_ALTERADOS	ATENCAO	Alteração de telefone, e-mail ou endereço (RF10)

3.3 Movimentações e retirada

Ação	Nível	Quando
TRANSFERENCIA_SOLICITADA	INFO	Requisição recebida
TRANSFERENCIA_EFETIVADA	INFO	Após COMMIT
TRANSFERENCIA_RECUSADA	ATENCAO	Risco alto, saldo insuficiente ou limite excedido
CHAVE_PIX_CRIADA / CHAVE_PIX_REMOVIDA	ATENCAO	Gestão de chaves (RF16)
RETIRADA_SOLICITADA	INFO	RF59
RETIRADA_LIBERADA	ATENCAO	Após 2FA válido (RF62)
RETIRADA_2FA_FALHOU	CRITICO	3 tentativas esgotadas
RETIRADA_CANCELADA / RETIRADA_ESTORNADA	ATENCAO	RF63
ESTORNO_REALIZADO	ATENCAO	RN10

3.4 Cartões e pagamentos

CARTAO_EMITIDO, CARTAO_BLOQUEADO, CARTAO_DESBLOQUEADO, LIMITE_ALTERADO, FATURA_CONTESTADA, CONTESTACAO_DEFERIDA, CONTESTACAO_INDEFERIDA, BOLETO_PAGO, COBRANCA_PIX_EMITIDA, COBRANCA_PIX_LIQUIDADADA, PAGAMENTO_AGENDADO.

3.5 Assinatura

Ação	Nível	Quando
ASSINATURA_CONTRATADA	INFO	RF65
PLANO_ALTERADO	ATENCAO	Upgrade ou downgrade (RF67)
ITEM_INCLUIDO	INFO	RF68
ITEM_RETIRADO	ATENCAO	Retirada de item — registra data e motivo (RN22, RF69)
COBRANCA_GERADA / COBRANCA_PAGA / COBRANCA_ESTORNADA	INFO	RF66
ASSINATURA_SUSPENSA	ATENCAO	Atraso > 5 dias (RN25)
ASSINATURA_CANCELADA	ATENCAO	RF72

3.6 Administração

Ação	Nível	Quando
USUARIO_INTERNO_CRIADO / EDITADO / DESATIVADO	ATENCAO	RF43
PERFIL_ALTERADO	CRITICO	RF44
PARAMETRO_ALTERADO	CRITICO	Tarifas, limites e tetos (RF49)
RELATORIO_GERADO	INFO	RF50
AUDITORIA_CONSULTADA	INFO	A consulta do auditor também é auditada (RN17)
AUDITORIA_EXPORTADA	ATENCAO	RF75

4. Modelo de dados

4.1 Tabela logs_auditoria (PostgreSQL)

Coluna	Tipo	Nulo	Descrição
id	bigserial	não	Chave primária sequencial (append-only)
criado_em	timestampz	não	Data/hora do evento (default now())
nivel	varchar(10)	não	INFO , ATENCAO , CRITICO
acao	varchar(60)	não	Código do evento (seção 3)
entidade	varchar(60)	sim	Tabela/classe afetada (contas , retiradas , assinaturas ...)
entidade_id	uuid	sim	Identificador do registro afetado
ator_id	uuid	sim	Usuário que executou (null = sistema/anonimo)
ator_perfil	varchar(20)	sim	Perfil no momento do evento
ator_nome	varchar(120)	sim	Nome congelado no momento do evento
dados_anteriores	jsonb	sim	Estado antes da operação
dados_novos	jsonb	sim	Estado depois da operação
metadados	jsonb	sim	Valor, canal, motivo, resultado da análise de risco
ip	inet	sim	Endereço IP de origem
user_agent	text	sim	Navegador/aplicativo
dispositivo_id	uuid	sim	Dispositivo confiável, quando houver
hash_registro	char(64)	não	SHA-256 do conteúdo do registro
hash_anterior	char(64)	sim	SHA-256 do registro anterior (encadeamento)

Índices

```
CREATE INDEX idx_auditoria_acao      ON logs_auditoria (acao);
CREATE INDEX idx_auditoria_ator      ON logs_auditoria (ator_id);
CREATE INDEX idx_auditoria_entidade  ON logs_auditoria (entidade, entidade_id);
CREATE INDEX idx_auditoria_periodo  ON logs_auditoria (criado_em DESC);
CREATE INDEX idx_auditoria_nivel     ON logs_auditoria (nivel) WHERE nivel <> 'INFO';
```

4.2 Migration (Laravel)

```
<?php
// database/migrations/2026_08_31_000001_create_logs_auditoria_table.php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('logs_auditoria', function (Blueprint $table) {
            $table->id();
            $table->timestampsTz(null, null)->useCurrent()->nullable(false);
            $table->string('nivel', 10);
            $table->string('acao', 60);
            $table->string('entidade', 60)->nullable();
            $table->uuid('entidade_id')->nullable();
            $table->uuid('ator_id')->nullable();
            $table->string('ator_perfil', 20)->nullable();
            $table->string('ator_nome', 120)->nullable();
            $table->jsonb('dados_anteriores')->nullable();
            $table->jsonb('dados_novos')->nullable();
            $table->jsonb('metadados')->nullable();
            $table->string('ip', 45)->nullable();
            $table->text('user_agent')->nullable();
            $table->uuid('dispositivo_id')->nullable();
            $table->char('hash_registro', 64);
            $table->char('hash_anterior', 64)->nullable();
        });

        // RF46 / RN27 – a trilha não admite UPDATE nem DELETE.
        DB::unprepared(<<<'SQL'
        CREATE OR REPLACE FUNCTION bloquear_alteracao_auditoria()
        RETURNS trigger LANGUAGE plpgsql AS $$
        BEGIN
            RAISE EXCEPTION
                'RF46: registros de auditoria sao imutaveis (operacao % negada)',
                TG_OP
            USING ERRCODE = 'restrict_violation';
        END;
        $$;

        CREATE TRIGGER trg_logs_auditoria_imutavel
        BEFORE UPDATE OR DELETE ON logs_auditoria
        FOR EACH ROW EXECUTE FUNCTION bloquear_alteracao_auditoria();
SQL);
    }

    public function down(): void
    {
        DB::unprepared('DROP TRIGGER IF EXISTS trg_logs_auditoria_imutavel ON logs_auditoria');
        DB::unprepared('DROP FUNCTION IF EXISTS bloquear_alteracao_auditoria()');
        Schema::dropIfExists('logs_auditoria');
    }
};
```

5. Registro do evento (código da aplicação)

5.1 Trait `RegistraAuditoria`

```
<?php
// app/Services/Auditoria/RegistraAuditoria.php
namespace App\Services\Auditoria;

use Illuminate\Support\Arr;
use Illuminate\Support\Facades\{Auth, Request};

trait RegistraAuditoria
{
    protected function registrarAuditoria(
        string $acao,
        ?object $entidade = null,
        array $antes = [],
        array $depois = [],
        array $metadados = [],
        string $nivel = 'INFO',
    ): void {
        $ator = Auth::user();

        // RNF43 – nunca registrar senha, código 2FA, token ou CVV.
        $antes = $this->mascarar($antes);
        $depois = $this->mascarar($depois);

        ProcessaAuditoria::dispatch([
            'nivel'          => $nivel,
            'acao'           => $acao,
            'entidade'       => $entidade?->getTable(),
            'entidade_id'    => $entidade?->getKey(),
            'ator_id'        => $ator?->id,
            'ator_perfil'    => $ator?->perfil,
            'ator_nome'      => $ator?->nome,
            'dados_anteriores' => $antes,
            'dados_novos'    => $depois,
            'metadados'      => $metadados,
            'ip'             => Request::ip(),
            'user_agent'     => Str::limit(Request::userAgent(), 250),
            'dispositivo_id' => Request::header('X-Device-Id'),
        ]->onQueue('auditoria'); // RNF41 – fora do caminho crítico
    }

    private function mascarar(array $dados): array
    {
        return Arr::map($dados, fn ($v, $k) => match (true) {
            in_array($k, ['senha', 'senha_hash', 'codigo', 'codigo_hash', 'token', 'cvv'], true)
                => '****', // RNF43
            in_array($k, ['cpf', 'cnpj'], true)
                => Mascarador::cpf((string) $v), // RNF08 / RNF13
            default => $v,
        });
    }
}
```

5.2 Job de gravação com hash encadeado

```
<?php
// app/Jobs/ProcessaAuditoria.php
namespace App\Jobs;

use Illuminate\Support\Facades\DB;
use Illuminate\Support\Str;

class ProcessaAuditoria implements ShouldQueue
{
    use Queueable, InteractsWithQueue;

    public function handle(): void
    {
        DB::transaction(function () {
            // Encadeamento: pega o hash do ultimo registro (lock para concorrência).
            $hashAnterior = DB::table('logs_auditoria')
                ->lockForUpdate()
                ->orderByDesc('id')
                ->value('hash_registro');

            $linha = [...$this->dados, 'hash_anterior' => $hashAnterior];
            $linha['hash_registro'] = $this->hash($linha);

            DB::table('logs_auditoria')->insert($linha);
        });
    }

    private function hash(array $linha): string
    {
        $canonico = json_encode(
            Arr::except($linha, ['hash_registro']),
            JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES | JSON_PRESERVE_ZERO_FRACTION
        );

        return hash('sha256', $canonico);
    }
}
```

5.3 Verificação de integridade (RNF42)

```
// app/Console/Commands/VerificarAuditoria.php
// Execução diária (schedule): php artisan auditoria:verificar
$anterior = null; $violacoes = 0;

DB::table('logs_auditoria')->orderBy('id')->chunk(500, function ($lote) use (&$anterior, &$violacoes)
{
    foreach ($lote as $r) {
        $esperado = hash('sha256', json_encode(Arr::except((array) $r, ['hash_registro'])));
        if ($esperado !== $r->hash_registro || $r->hash_anterior !== $anterior) {
            $violacoes++;
            Log::critical('TRILHA_ADULTERADA', ['id' => $r->id]);
        }
        $anterior = $r->hash_registro;
    }
});
```

6. Exemplo de registro

6.1 Retirada liberada com 2FA

```
{
  "id": 10482,
  "criado_em": "2026-08-31T21:14:07-03:00",
  "nivel": "ATENCAO",
  "acao": "RETIRADA_LIBERADA",
  "entidade": "retiradas",
  "entidade_id": "9f1c2a44-7d3e-4b0a-9c51-2f8b6de1a0c3",
  "ator_id": "c3f9a1d2-5b0e-4c8a-8d71-6e2f9a4b1c55",
  "ator_perfil": "CLIENTE",
  "ator_nome": "Maria Silva",
  "dados_anteriores": {
    "status": "AGUARDANDO_2FA",
    "codigo_retirada": null,
    "liberada_em": null
  },
  "dados_novos": {
    "status": "LIBERADA",
    "codigo_retirada": "4 8 1 9 0 7",
    "liberada_em": "2026-08-31T21:14:07-03:00"
  },
  "metadados": {
    "valor": "850.00",
    "canal": "CAIXA_PARCEIRO",
    "exigiu_2fa": true,
    "motivo_2fa": ["valor_acima_30pct_limite", "periodo_noturno"],
    "tentativas_2fa": 1,
    "transacao_id": "b7d0e3f1-2a4c-4f8b-9e13-5c7a2d8f6b10",
    "lancamentos": [
      { "conta": "0001-00045782-1", "natureza": "D", "valor": "850.00" },
      { "conta": "0001-00000001-9", "natureza": "C", "valor": "850.00" }
    ]
  },
  "ip": "177.67.82.14",
  "user_agent": "Fluxo/1.0 (Android 13; SM-A145M)",
  "dispositivo_id": "e7b2c9a0-3f1d-4e6a-9b82-1c5d7f3a8e24",
  "hash_anterior": "3f9a1c...",
  "hash_registro": "a71c0e..."
}
```

6.2 Retirada de item da assinatura (o caso da "retirada de coisas")

```
{
  "acao": "ITEM_RETIRADO",
  "entidade": "itens_assinatura",
  "nivel": "ATENCAO",
  "dados_anteriores": { "quantidade": 2, "data_exclusao": null },
  "dados_novos": {
    "quantidade": 2,
    "data_exclusao": "2026-08-31",
    "motivo_exclusao": "Cliente reduziu o plano - nao utiliza mais o relatorio gerencial"
  },
  "metadados": {
    "item": "Relatorio gerencial mensal",
    "valor_unitario": "29.90",
    "impacto_cobranca": "-59.80",
    "cobranca_recalculada": "competencia 2026-08"
  }
}
```

Note que **não há DELETE**: o registro continua existindo com `data_exclusao` preenchida, e a trilha guarda o antes, o depois e o motivo (RN22).

7. Consulta pelo auditor

7.1 Endpoint

```
GET /api/v1/auditoria?ator_id=&acao=&entidade=&entidade_id=&nivel=&de=&ate=&q=&page=
```

Permissão: **Policy** — apenas perfis AUDITOR e ADMINISTRADOR (RF44, RF48). O acesso é somente leitura e a própria consulta gera AUDITORIA_CONSULTADA (RN17).

7.2 Tela (Portal Admin)

Elemento	Comportamento
Filtros	Autor, ação, entidade, nível, período e texto livre
Ordenação	Mais recentes primeiro, paginação de 50 (RNF05)
Detalhe	Modal com antes/depois em diff e os metadados completos
Exportação	CSV com os mesmos filtros aplicados (RF75)
Selo de integridade	Exibe o resultado da última verificação de hash encadeado

7.3 Consulta SQL de referência

```
SELECT criado_em, acao, entidade, entidade_id, ator_nome, ator_perfil, ip, nivel
FROM logs_auditoria
WHERE criado_em >= '2026-08-01' AND criado_em < '2026-09-01'
AND acao IN ('RETIRADA_LIBERADA', 'RETIRADA_2FA_FALHOU')
ORDER BY criado_em DESC
LIMIT 50;
```

8. Retenção, LGPD e segurança

Tema	Política
Retenção	5 anos para operações financeiras (RNF44); 1 ano para eventos de nível INFO de navegação
Minimização	Somente campos necessários; CPF e dados de cartão mascarados
Proibição absoluta	Senha, hash de senha, código 2FA, token de sessão e CVV nunca entram no payload (RNF43)
Direito de exclusão (LGPD)	Dados pessoais do titular são anonimizados; os registros de auditoria são mantidos , pois a finalidade é jurídica e de segurança — o titular é informado dessa base legal no termo de uso
Acesso	Somente leitura para auditor; gravação apenas pelo job, nunca por usuário
Backup	pg_dump semanal (RNF19); a trilha é recriada a partir do dump, preservando o encadeamento de hash

9. Testes previstos

#	Teste	Resultado esperado
T1	Registrar um evento e consultar	Registro presente com todos os campos

#	Teste	Resultado esperado
T2	Tentar UPDATE na trilha	Exceção do trigger (restrict_violation)
T3	Tentar DELETE na trilha	Exceção do trigger
T4	Alterar um registro diretamente no banco e rodar auditoria:verificar	Violação detectada e registrada
T5	Realizar login, Pix e retirada	100% dos eventos esperados presentes
T6	Auditor consulta a trilha	AUDITORIA_CONSULTADA gerado
T7	Cliente tenta acessar /api/v1/auditoria	403 Forbidden
T8	Medir latência da transferência com e sem auditoria	Diferença < 100 ms (RNF41)
T9	Verificar payload de uma retirada	Nenhum campo sensível presente
T10	Exportar CSV com filtros	Arquivo com o mesmo total da consulta

10. Rastreabilidade

Diagrama	Relação com o log de auditoria
SEQ-01	CADASTRO_SOLICITADO e CADASTRO_APROVADO
SEQ-02	LOGIN_FALHOU , LOGIN_2FA_SUCESSO , 2FA_FALHOU
SEQ-03	TRANSFERENCIA_EFETIVADA / TRANSFERENCIA_RECUSADA
SEQ-04	RETIRADA_SOLICITADA , RETIRADA_2FA_FALHOU , RETIRADA_LIBERADA
SEQ-05	BOLETO_PAGO
SEQ-06	Registro e consulta da própria trilha
CLS-01 / CLS-02	Classe LogAuditoria («append-only»)
ATV-02 / ATV-03 / ATV-05	Pontos do fluxo em que cada evento é disparado

Registro de alterações

Versão	Data	Alteração
1.0	31/08/2026	Versão inicial: matriz de eventos, modelo de dados, migration, job com hash encadeado, consulta e políticas